Java Annotations and Reflection

#	Scenario Title / Use Case	Level	Target Role
1	Defining custom annotations to tag method metadata	L2	Java Developer / Framework Engineer
2	Annotation processors for automated code generation	L4	Compiler Engineer / Framework Dev
3	Accessing private class members via reflection for tests	L2	Test Engineer / Java Dev
4	Invoking methods dynamically by name	L2	Integration Engineer
5	Inspecting class structure at runtime	L3	Tooling Engineer / Framework Author
6	Adding default values to custom annotations	L2	Java Developer / API Designer
7	Retrieving custom annotation data at runtime	L3	Backend Engineer / Framework Dev
8	Ensuring correct method overriding in subclasses	L1	Junior Java Developer
9	Recognizing & avoiding deprecated methods	L1	Maintenance Engineer
10	Selectively suppressing compiler warnings	L1	Java Developer
11	Creating annotations to identify test methods	L2	Test Framework Engineer
12	Using annotations to mark methods for logging	L3	Backend Engineer
13	Designing annotations usable on multiple element types	L2	Framework Developer
14	Instantiating classes dynamically by name	L2	Plugin Developer
15	Using qualifier annotations for dependency injection	L4	Backend Architect
16	Scheduling method execution via annotation	L3	Scheduler Dev
17	Performance monitoring with annotations	L3	Performance Engineer
18	Environment-based method access control	L4	Platform Engineer

19	Tracking how often methods are called	L3	Observability Engineer
20	Circuit Breaker pattern via annotations	L4	Resilience Architect
21	Automating method entry/exit logging	L3	AOP Specialist
22	Automatic exception logging with annotations	L3	Error-Handling Specialist
23	Retrying transient failures via annotation	L4	Backend Resilience Engineer
24	Rate-limiting method calls	L4	Backend Engineer
25	Validating fields using custom annotations	L2	Java Developer
26	Combining annotations for multiple rules	L3	Framework Engineer
27	Annotation-based service discovery	L4	Backend Architect
28	Dynamic runtime class loading	L4	System Engineer
29	Generic annotation reader utility	L2	Java Developer
30	Adding logging via dynamic proxy	L3	Backend Engineer / AOP Developer
31	Designing a REST API router using annotations	L4	Backend Engineer / Framework Dev
32	Building transaction management with custom annotations	L4	Backend Engineer / Architect
33	Role-based access control using annotations	L3	Security Engineer / Backend Dev
34	Method result caching using annotations	L3	Backend Engineer
35	Defining how long annotations are retained	L1	Java Developer
36	Using reflection to retrieve all methods of a class	L1	Java Developer
37	Enforcing non-null fields with custom annotation	L2	Java Developer / Validator
38	Adding metadata to fields for documentation	L2	Framework / Doc Engineer
39	Validation annotation with parameters (max length)	L2	Java Developer

40	Creating instances using reflection (default ctor)	L3	Java Developer
41	Enforcing deprecated API usage warnings	L1	Java Developer
42	Custom annotation to track API versioning	L3	Backend Engineer
43	Runtime processing of custom annotations for authorization	L3	Security Engineer
44	Listing all fields and their annotations in a class	L2	Tooling Engineer / ORM Dev
45	Dynamically invoking private methods for testing	L2	Test Engineer
46	Logger aspect detecting overridden methods	L4	AOP / Performance Engineer
47	Custom annotation with multiple parameters and defaults	L2	Java Developer
48	Suppressing specific warnings in legacy code	L1	Maintenance Engineer
49	Annotation processor to generate documentation	L4	Compiler Engineer / Tooling Dev
50	Dynamic proxy for logging interface calls	L3	Backend Engineer
51	Modifying field values dynamically for configuration	L3	Backend Engineer / DevOps
52	Detecting and invoking methods annotated with @Init	L3	Backend Engineer / Lifecycle Manager
53	Runtime analysis of class method signatures and annotations	L3	Tooling Engineer
54	Handling multiple constructors and choosing one at runtime	L4	Backend Engineer
55	Custom annotation to mark deprecated fields	L3	Backend Engineer / Security Auditor

Scenario 1: Defining custom annotations to tag method metadata

You want a structured way to label methods with information like complexity, priority, and author, instead of informal comments.

Answer:

We define a custom annotation interface with attributes such as author, complexity, and priority. By marking it with @Retention(RetentionPolicy.RUNTIME), the annotation will be preserved at runtime and can be accessed with reflection. Using @Target(ElementType.METHOD) ensures it's only applied to methods.

Code:

```
import java.lang.annotation.*;

// Define annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Metadata {
    String author();
    String complexity();
    int priority();
}

class Sample {

    @Metadata(author = "Alice", complexity = "Medium", priority = 1)
    public void performTask() {
        System.out.println("Executing a task with metadata");
    }

    @Metadata(author = "Bob", complexity = "High", priority = 2)
    public void criticalOperation() {
        System.out.println("Performing a critical operation");
    }
}
```

Reasoning to interviewer:

- Custom annotations replace comments with structured, type-safe metadata.
- Runtime retention allows tools or frameworks to dynamically read and act on this data.

- This enables cleaner maintainability and automated processing of methods based on metadata.

Future scope:

- Add default values for attributes to reduce boilerplate.
- Combine with reflection to filter and execute methods by priority.
- Use similar patterns for security (@Secured(role="ADMIN")) or validation (@NotNull).

Scenario 2: Using annotation processors for automated code generation

You want to eliminate repetitive coding tasks, like adding logging or validation, by automatically generating supporting code during compilation.

Answer:

Java's annotation processing API allows us to detect custom annotations during compile time and generate code. By extending AbstractProcessor, we can scan for annotated elements and create boilerplate or helper classes automatically, ensuring consistency and reducing manual effort.

Code:

```
import javax.annotation.processing.*;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.*;
import java.util.Set;

// Custom annotation
@Retention(RetentionPolicy.SOURCE)
@Target(ElementType.TYPE)
@interface Trackable {}

// Annotation processor
@SupportedAnnotationTypes("Trackable")
@SupportedSourceVersion(SourceVersion.RELEASE_17) // Adjust version if
needed
public class LoggingProcessor extends AbstractProcessor {

    @Override
    public boolean process(Set<? extends TypeElement> annotations,
RoundEnvironment env) {
        for (Element element : env.getElementsAnnotatedWith(Trackable.class)) {
            String className = element.getSimpleName().toString();
            System.out.println("Generating logger for class: " + className);
            // In practice, use the Filer API to write new Java source files
        }
        return true;
    }
}
```

Reasoning to interviewer:

- Annotation processors run at compile time, scanning code for annotations.

- They enable boilerplate reduction — frameworks like Lombok and MapStruct rely on this.
- @SupportedAnnotationTypes and @SupportedSourceVersion define what the processor handles.

Future scope:

- Use the Filer API to generate .java files dynamically.
- Chain multiple processors for validation, logging, and code-gen.
- Integrate with build tools (Maven/Gradle) so generated sources compile automatically.

Scenario 3: Accessing private class members using reflection for testing

Sometimes we need to validate or adjust internal fields in tests without altering the original class (e.g., verifying state of a framework class).

Answer:

Reflection lets us bypass Java's access control with setAccessible(true). By retrieving a Field object from the class, we can read or update private fields on an instance. This is common in testing when we need to inspect internal state without changing production code.

Code:

```
import java.lang.reflect.Field;

class Person {
    private String name = "John Doe";
}

public class ReflectionExample {
    public static void main(String[] args) throws Exception {
        Person person = new Person();

        Field privateField = Person.class.getDeclaredField("name");
        privateField.setAccessible(true);  // bypass private access

        System.out.println("Initial Name: " + privateField.get(person));

        privateField.set(person, "Jane Doe"); // modify private field
        System.out.println("Changed Name: " + privateField.get(person));
    }
}
```

Reasoning to interviewer:

- Reflection gives runtime control over class internals — useful in testing frameworks (JUnit, Mockito).
- setAccessible(true) overrides Java's access checks, letting us read or write private fields.

- This should be used carefully, as it can break encapsulation and may not work under strict security managers.

Future scope:

- Wrap reflective access in utility methods for cleaner test code.
- Use frameworks like Spring or Jackson which already leverage reflection for DI and serialization.
- Combine with annotations to drive test validations (e.g., auto-inject values).

Scenario 4: Invoking methods dynamically by their names

In plugin systems or dynamic integrations, method names may only be known at runtime.

Answer:

Using reflection, we fetch a Method object with getMethod(methodName, paramTypes), then invoke it on an object instance with invoke(). This lets us call arbitrary methods dynamically, which is powerful for extensibility.

Code:

```
import java.lang.reflect.Method;

class Messenger {
    public void sayHello(String name) {
        System.out.println("Hi, " + name);
    }
}

public class DynamicInvoker {
    public static void main(String[] args) throws Exception {
        Messenger messenger = new Messenger();

        // Fetch method by name and parameter type
        Method method = Messenger.class.getMethod("sayHello", String.class);

        // Invoke dynamically at runtime
        method.invoke(messenger, "Alice");
    }
}
```

Reasoning to interviewer:

- Reflection enables late binding: we don’t need compile-time knowledge of method names.
- This is the basis for frameworks like Spring, Hibernate, and dynamic RPC systems.
- invoke() accepts both the target instance and arguments, so it supports overloads.

Future scope:

- Use caching of Method handles to improve performance.
- Switch to MethodHandle / LambdaMetafactory (introduced in Java 7/8) for faster dynamic invocation.
- Apply in plugin architectures, scripting engines, or annotation-driven frameworks.

Scenario 5: Inspecting class components at runtime using reflection

You want to build a small introspection tool that prints all fields, methods, and constructors of any class to help with debugging or documentation.

Answer:

I'll use the core Reflection API: Class<?> plus getDeclaredFields(), getDeclaredMethods(), and getDeclaredConstructors(). I'll also print modifiers, parameter types, and return types so the output is genuinely useful. getDeclared* shows everything declared in the class (including private members); get* shows only public members, including inherited ones.

Code:

```
import java.lang.reflect.*;
import java.util.Arrays;
import java.util.stream.Collectors;

class DemoClass {
    private int number;
    public DemoClass() {}
    private DemoClass(int number) { this.number = number; }
    public void show() { System.out.println("Show method"); }
    protected String greet(String name) { return "Hello " + name; }
}

public class ClassInspector {
    public static void main(String[] args) {
        inspect(DemoClass.class);
    }

    static void inspect(Class<?> clazz) {
        System.out.println("Class: " + clazz.getName());

        System.out.println("\nFields:");
        for (Field f : clazz.getDeclaredFields()) {
            String mods = Modifier.toString(f.getModifiers());
            System.out.printf(" * %s %s %s%n",
                    mods.isEmpty() ? "(package-private)" : mods,
                    f.getType().getSimpleName(),
                    f.getName());
        }

        System.out.println("\nConstructors:");
        for (Constructor<?> c : clazz.getDeclaredConstructors()) {
```

```
            String mods = Modifier.toString(c.getModifiers());
            String params = Arrays.stream(c.getParameterTypes())
                                  .map(Class::getSimpleName)
                                  .collect(Collectors.joining(", "));
            System.out.printf(" * %s %s(%s)%n",
                  mods.isEmpty() ? "(package-private)" : mods,
                  clazz.getSimpleName(),
                  params);
        }

        System.out.println("\nMethods:");
        for (Method m : clazz.getDeclaredMethods()) {
            String mods = Modifier.toString(m.getModifiers());
            String params = Arrays.stream(m.getParameterTypes())
                                  .map(Class::getSimpleName)
                                  .collect(Collectors.joining(", "));
            System.out.printf(" * %s %s %s(%s)%n",
                  mods.isEmpty() ? "(package-private)" : mods,
                  m.getReturnType().getSimpleName(),
                  m.getName(),
                  params);
        }
    }
}
```

Reasoning to interviewer:

- getDeclared* exposes the full picture of a class (including private/package-private) which is ideal for documentation tools.
- Printing modifiers, parameter types, and return types makes the output actionable, not just a list of names.
- This can be wrapped into a utility that accepts any Class<?> (or even a fully qualified name via Class.forName) to introspect arbitrary types.

Future scope:

- Include annotations on classes/members (getAnnotations()) to document metadata.
- Add inheritance traversal to include members from superclasses and interfaces.
- Export to JSON/Markdown to integrate with internal docs.
- Gate private access with a flag for environments that restrict reflective access.

Scenario 6: Adding default values to custom annotations

You want optional annotation elements (like priority and reviewer) to have sensible defaults so callers can omit them.

Answer:

In the annotation interface, declare elements with default values. When the annotation is used without those elements, the defaults apply automatically. I'll set runtime retention so tools can read them during execution.

Code:

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Highlight {
    String reviewer() default "No Reviewer";
    int priority() default 5; // 1 = highest urgency, 5 = normal
}

class Job {

    @Highlight // uses defaults: reviewer="No Reviewer", priority=5
    public void importantTask() {
        System.out.println("Running important task");
    }

    @Highlight(reviewer = "Priya", priority = 2) // override defaults
    public void urgentTask() {
        System.out.println("Running urgent task");
    }
}
```

Reasoning to interviewer:

- Defaults reduce noise at call sites while keeping the annotation expressive when overrides are needed.
- @Retention(RUNTIME) makes the values accessible via reflection (e.g., a scheduler could sort tasks by priority).
- Keeping elements optional avoids breaking callers if you add new elements later (with defaults).

Future scope:

- Add enum elements (e.g., Severity { LOW, MEDIUM, HIGH }) for stronger typing than plain ints/strings.
- Combine with reflection to build dashboards or to route methods to different queues based on priority.
- Create meta-annotations or compose multiple custom annotations for richer policies (e.g., @Audited @Highlight).

Scenario 7: Retrieving custom annotation data at runtime

You want to configure behavior (like logging) by reading values from custom annotations on methods.

Answer:

At runtime, I'll reflect on the method, call getAnnotation(MyAnno.class), and then read its attributes. Since the annotation has @Retention(RetentionPolicy.RUNTIME), the values are available during execution. I can then route behavior—for example, set the logger level or prefix messages—based on those values.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Method;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Logging {
    String level() default "INFO";
    String message() default "Executing";
}

class Processor {
    @Logging(level = "DEBUG", message = "Starting process")
    public void run() {
        System.out.println("Process running");
    }
}

class AnnotationReader {
    public static void main(String[] args) throws Exception {
        Method m = Processor.class.getMethod("run");
        Logging meta = m.getAnnotation(Logging.class);

        if (meta != null) {
            // dynamic behavior driven by annotation values
            System.out.println("[level=" + meta.level() + "] " + meta.message());
            new Processor().run();
        } else {
            new Processor().run();
        }
    }
}
```

Reasoning to interviewer:

- @Retention(RUNTIME) is essential; without it, annotations aren't visible via reflection.
- getAnnotation returns a proxied annotation instance; calling meta.level()/meta.message() is cheap and type-safe.
- This pattern decouples configuration from code logic—great for cross-cutting concerns like logging, tracing, or metrics.

Future scope:

- Add method-level defaults and class-level overrides (check class annotation if method lacks one).
- Cache annotation lookups per Method to avoid repeated reflection overhead.
- Combine with AOP (e.g., a method interceptor) so logging happens automatically around annotated methods.

Scenario 8: Ensuring correct method overriding in subclasses

You want to prevent silent bugs where a method in a subclass fails to override due to a typo or wrong signature.

Answer:

I'll use @Override on every intended override. The compiler enforces that the annotated method exactly matches a superclass/interface method. If it doesn't, compilation fails—so we catch the issue early.

Code:

```
class Animal {
    void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() { // compiler verifies this is a real override
        System.out.println("Bark");
    }

    // If I accidentally wrote:
    // @Override
    // void makeSounds() {} // <-- compile error: method does not override or
implement
}
```

Reasoning to interviewer:

- @Override is a low-friction safety net that prevents accidental overloading instead of overriding.
- It also improves readability: the intent to override is explicit, which helps in code reviews and refactors.
- Works with interface methods too; using it consistently across the codebase reduces subtle runtime surprises.

Future scope:

- Enforce @Override via static analysis or IDE inspections to keep the rule consistent.
- Combine with @FunctionalInterface for SAM interfaces—compiler guarantees exactly one abstract method.
- In large refactors, search for methods lacking @Override to find potential mis-overrides quickly.

Scenario 9: Recognizing and avoiding deprecated methods

When you see code calling outdated methods, it signals potential maintenance and compatibility risks.

Answer:

Java uses the @Deprecated annotation to mark methods, constructors, or classes that shouldn't be used anymore. The compiler and most IDEs will highlight these usages with warnings. Deprecated methods still work, but they might be removed in future releases. The right practice is to look at the Javadoc, which usually suggests a replacement, and refactor the code to use the newer API. This keeps the codebase healthier and reduces technical debt.

Code:

```
class Legacy {
    @Deprecated
    void oldMethod() {  // flagged by compiler and IDE
        System.out.println("Deprecated method");
    }

    void newMethod() {  // suggested alternative
        System.out.println("Preferred method");
    }
}

class Demo {
    public static void main(String[] args) {
        Legacy legacy = new Legacy();
        legacy.oldMethod(); // warning: deprecated
        legacy.newMethod(); // safe to use
    }
}
```

Reasoning to interviewer:

- @Deprecated is a signal, not a blocker: it compiles but warns.
- Documentation often contains @deprecated Javadoc with a "use-this-instead" suggestion.
- By proactively replacing deprecated calls, we avoid surprises when upgrading libraries or JDK versions.

Future scope:

- Configure build tools (Maven, Gradle) to treat deprecation warnings as errors, enforcing cleanup.
- Automate scanning for deprecated API usage via static analysis (SonarQube, Checkstyle).
- Train team to check release notes for newly deprecated APIs during upgrades.

Scenario 10: Selectively suppressing compiler warnings in legacy code

Legacy codebases often throw many “unchecked” or “deprecation” warnings, making real problems hard to spot.

Answer:

The @SuppressWarnings annotation lets you silence only specific categories of warnings. For example, "unchecked" for raw type casts, or "deprecation" for calls to deprecated APIs. This should be applied in the smallest possible scope (like a single method), not on entire classes, so that other valuable warnings are not hidden.

Code:

```
import java.util.List;

class LegacyProcessor {

    @SuppressWarnings("unchecked") // suppress only this warning here
    void processList(List list) {
        // raw List is being cast; normally triggers an unchecked warning
        List<String> stringList = (List<String>) list;
        stringList.forEach(System.out::println);
    }

    @SuppressWarnings("deprecation")
    void useDeprecated() {
        Legacy legacy = new Legacy();
        legacy.oldMethod(); // suppresses warning about deprecation
    }
}
```

Reasoning to interviewer:

- @SuppressWarnings makes the compiler less noisy, so developers focus on real issues.
- Targeting it narrowly avoids hiding unrelated warnings in other parts of the code.
- Common keys: "unchecked", "deprecation", "rawtypes".

Future scope:

- Use @SuppressWarnings as a temporary bridge while refactoring legacy code.
- Integrate static analysis to track and eventually remove suppressed blocks.
- Apply IDE inspections to ensure suppressions are not abused across whole classes.

Scenario 11: Creating annotations to identify test methods

You want to design your own lightweight testing framework where methods can be marked as test cases. Instead of relying on JUnit or TestNG, you'll use a custom annotation that helps the framework detect and execute test methods dynamically.

Answer:

To achieve this, I'll define a custom annotation using @interface. It must be retained at runtime (@Retention(RetentionPolicy.RUNTIME)) so that the testing framework can read it using reflection. By restricting its usage to methods with @Target(ElementType.METHOD), developers can't accidentally place it on fields or classes.

When writing tests, developers simply apply this annotation on any method that should be executed as part of the test suite. At runtime, the test runner will scan through all methods of a class, check if the annotation is present, and then invoke those methods dynamically. This avoids the need for naming conventions or external configuration and provides a clear, declarative way of tagging tests.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Method;

// Step 1: Define a custom annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface TestMethod {}

// Step 2: Write a sample test class
class SampleTests {

    @TestMethod
    public void testExample() {
        System.out.println("Executing testExample");
    }

    @TestMethod
    public void testAnotherCase() {
        System.out.println("Executing testAnotherCase");
    }

    public void helperMethod() {
        // Not marked, so this won't run
        System.out.println("Helper method, not a test");
```

```
    }
}

// Step 3: Implement a simple test executor
public class TestExecutor {
    public static void main(String[] args) throws Exception {
        SampleTests tests = new SampleTests();

        for (Method m : SampleTests.class.getDeclaredMethods()) {
            if (m.isAnnotationPresent(TestMethod.class)) {
                System.out.println("Running test: " + m.getName());
                m.invoke(tests);
            }
        }
    }
}
```

Reasoning to interviewer:

- This approach mirrors how real frameworks like JUnit or TestNG operate internally.
- The @TestMethod annotation provides clarity—developers don't rely on method naming conventions like testSomething(). Instead, they explicitly mark test cases.
- Reflection makes it possible to automatically discover and execute tests without manual registration.
- This design also encourages separation of concerns: developers write tests, and the framework handles orchestration.

Future scope:

- Add attributes to the annotation, such as priority() or expectedException(), to provide richer metadata about each test.
- Build a test runner that can handle failures gracefully by catching exceptions and reporting pass/fail results.
- Support grouping or categorization, for example @TestMethod(group="integration"), allowing selective execution of tests.
- Extend the runner to scan entire packages for test classes instead of a single class, turning this into a fully functional lightweight testing framework.

Scenario 12: Using annotations to mark methods for logging

In many applications, especially large enterprise systems, repetitive logging statements clutter business logic. A cleaner approach is to mark only those methods that need logging and let a centralized logging mechanism handle it automatically.

Answer:

To do this, I'll define a custom annotation called @Loggable. With @Retention(RetentionPolicy.RUNTIME) and @Target(ElementType.METHOD), this annotation can be applied to methods and remain accessible during runtime. The framework or a manager class will then use reflection to inspect methods, check for this annotation, and inject logging behavior before or after the actual method execution.

This makes logging declarative rather than imperative. Developers focus on business logic while the framework ensures logs are consistently applied wherever needed. This approach is similar in spirit to how Aspect-Oriented Programming (AOP) works in frameworks like Spring.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Method;

// Step 1: Define custom annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Loggable {}

// Step 2: Apply annotation on critical methods
class Service {

    @Loggable
    public void criticalOperation() {
        System.out.println("Performing critical operation");
    }

    public void regularOperation() {
        System.out.println("Performing regular operation");
    }
}
```

```
// Step 3: Build a logging manager that inspects and executes methods
public class LoggingManager {
    public static void main(String[] args) throws Exception {
        Service service = new Service();

        for (Method method : service.getClass().getDeclaredMethods()) {
            if (method.isAnnotationPresent(Loggable.class)) {
                System.out.println("LOG: Calling " + method.getName());
                long startTime = System.currentTimeMillis();

                method.invoke(service); // execute annotated method

                long duration = System.currentTimeMillis() - startTime;
                System.out.println("LOG: " + method.getName() + " executed in " +
duration + "ms");
            }
        }

        // Notice: regularOperation() is not executed or logged here
    }
}
```

Reasoning to interviewer:

- By marking methods with @Loggable, we cleanly separate logging from core logic.
- Reflection allows dynamic interception of annotated methods without hardcoding their names.
- This design reduces boilerplate code, increases maintainability, and ensures consistent logging across critical operations.
- It also aligns with cross-cutting concerns handling, a key concept in enterprise Java.

Future scope:

- Expand the @Loggable annotation with attributes, like level() (INFO, DEBUG, ERROR), so developers can control log verbosity.
- Integrate with logging frameworks such as SLF4J or Log4j for richer logging features (structured logs, appenders, external configuration).
- Implement method interception dynamically using proxies or AOP libraries, so logging is applied transparently without modifying runner code.

- Extend the mechanism to support before/after advice, enabling monitoring, security checks, or transaction management in addition to logging.

Scenario 13: Designing annotations usable on multiple element types

You want a single annotation that can flexibly mark both classes and methods for configuration, instead of defining two separate annotations.

Answer:

When defining an annotation, the @Target meta-annotation specifies where it can be applied. By passing an array of ElementType values, you can allow the same annotation to work on multiple program elements. For example, ElementType.TYPE covers classes, interfaces, and enums, while ElementType.METHOD covers individual methods. With both included, one annotation can be applied in either place.

Code:

```
import java.lang.annotation.*;

// Annotation that can be placed on classes AND methods
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
@interface Configurable {}

// Example usage
@Configurable
class ConfigClass {

    @Configurable
    public void initialize() {
        System.out.println("Initialization logic");
    }
}
```

Reasoning to interviewer:

- Using multiple ElementTypes in @Target provides flexibility without duplicating annotation definitions.
- This is useful when metadata or configuration applies at both the class and method level.

- Reflection can then detect the same annotation across different contexts, simplifying processing logic in frameworks or tooling.

Future scope:

- Extend the annotation with attributes (e.g., String value() default "";) to hold configuration parameters.
- Allow annotations at even more levels, like ElementType.FIELD or ElementType.PARAMETER, for deeper flexibility.
- Combine with annotation processing or reflection to build generic configuration frameworks.

Scenario 14: Instantiating classes by their name using reflection

You need to create objects dynamically when only a class name string is known, such as in plugin systems, configuration-driven architectures, or script engines.

Answer:

Reflection allows dynamic class loading using Class.forName("fully.qualified.ClassName"). Once the Class<?> object is obtained, you can create a new instance using the constructor API. The typical pattern is to call getConstructor().newInstance() for a no-argument constructor. After instantiation, you can invoke methods reflectively.

Code:

```
class DynamicExample {
    public void display() {
        System.out.println("Instance created dynamically");
    }
}

public class ReflectionDemo {
    public static void main(String[] args) throws Exception {
        // Load class by fully qualified name
        Class<?> clazz = Class.forName("DynamicExample");

        // Create a new instance using the no-argument constructor
        Object obj = clazz.getConstructor().newInstance();

        // Invoke a method dynamically
        clazz.getMethod("display").invoke(obj);
    }
}
```

Reasoning to interviewer:

- Class.forName() is the key to dynamic loading when class names are strings from config files, DB entries, or network requests.
- getConstructor().newInstance() ensures you're using the default constructor safely.
- Once instantiated, methods can be invoked reflectively, making this approach central to plugin systems and frameworks.

- This design is widely used in dependency injection containers, JDBC drivers, and classpath scanning frameworks.

Future scope:

- Handle constructors with parameters using getConstructor(paramTypes).newInstance(args).
- Use ClassLoader directly for loading classes from custom sources like JARs or remote modules.
- Cache loaded classes/instances to avoid performance overhead in high-frequency calls.
- Integrate with annotation scanning to auto-discover and instantiate annotated classes dynamically.

Scenario 15: Using qualifier annotations for dependency injection with multiple implementations

You have multiple implementations of the same interface and want to inject a specific one depending on where it's used (e.g., "Laser" printer in billing, "Inkjet" in marketing).

Answer:

I'll define a @Qualifier annotation that carries a discriminator (like a short name). Fields (or constructor parameters) that need injection will be annotated with @Qualifier("Laser"). The injector will:

1. scan the target's fields (or chosen constructor),
2. read the qualifier value,
3. resolve the matching implementation from a registry, and
4. set the field (or pass the dependency into the constructor).
   This keeps the wiring declarative and avoids brittle if/else logic strewn across the code.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.HashMap;
import java.util.Map;

// 1) Qualifier annotation usable on fields AND parameters
(constructor/setter DI)
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.FIELD, ElementType.PARAMETER})
@interface Qualifier {
    String value();
}

// 2) Service contract + implementations
interface Printer { void print(); }

class LaserPrinter implements Printer {
    public void print() { System.out.println("Using Laser Printer"); }
}
```

```
class InkjetPrinter implements Printer {
    public void print() { System.out.println("Using Inkjet Printer"); }
}

// 3) Simple registry for implementations (could be replaced by
ServiceLoader, etc.)
class BeanRegistry {
    private final Map<String, Printer> printers = new HashMap<>();
    public BeanRegistry() {
        printers.put("Laser", new LaserPrinter());
        printers.put("Inkjet", new InkjetPrinter());
    }
    public Printer getPrinter(String name) {
        Printer p = printers.get(name);
        if (p == null) throw new IllegalArgumentException("Unknown printer: "
+ name);
        return p;
    }
}

// 4a) Field injection example
class PrintManagerField {
    @Qualifier("Laser")
    private Printer printer;  // injected by qualifier

    public void printDocument() { printer.print(); }
}

// 4b) Constructor injection example (preferred for immutability/testability)
class PrintManagerCtor {
    private final Printer printer;
    public PrintManagerCtor(@Qualifier("Inkjet") Printer printer) {
        this.printer = printer;
    }
    public void printDocument() { printer.print(); }
}

// 5) Minimal injector supporting fields + constructor injection
class Injector {
    private final BeanRegistry registry = new BeanRegistry();

    public <T> T create(Class<T> type) {
        try {
            // Prefer the "greediest" constructor (more parameters) to support
ctor DI
            Constructor<?> chosen = null;
            for (Constructor<?> c : type.getDeclaredConstructors()) {
                if (chosen == null || c.getParameterCount() >
chosen.getParameterCount()) chosen = c;
```

```
            }
            if (chosen == null || chosen.getParameterCount() == 0) {
                // No-arg path, then do field injection
                T instance = type.getDeclaredConstructor().newInstance();
                injectFields(instance);
                return instance;
            } else {
                Object[] args = resolveParameters(chosen);
                @SuppressWarnings("unchecked")
                T instance = (T) chosen.newInstance(args);
                injectFields(instance);
                return instance;
            }
        } catch (Exception e) {
            throw new RuntimeException("Injection failed for " +
type.getName(), e);
        }
    }

    private Object[] resolveParameters(Constructor<?> ctor) {
        Parameter[] params = ctor.getParameters();
        Object[] args = new Object[params.length];
        for (int i = 0; i < params.length; i++) {
            Parameter p = params[i];
            if (p.isAnnotationPresent(Qualifier.class) && p.getType() ==
Printer.class) {
                String name = p.getAnnotation(Qualifier.class).value();
                args[i] = registry.getPrinter(name);
            } else {
                throw new IllegalStateException("Unsupported parameter: " + p);
            }
        }
        return args;
    }

    private void injectFields(Object target) throws IllegalAccessException {
        for (Field field : target.getClass().getDeclaredFields()) {
            if (field.isAnnotationPresent(Qualifier.class) && field.getType() ==
Printer.class) {
                String name = field.getAnnotation(Qualifier.class).value();
                field.setAccessible(true);
                field.set(target, registry.getPrinter(name));
            }
        }
    }
}

// 6) Demo
public class MainApp {
    public static void main(String[] args) {
```

```
        Injector injector = new Injector();

        PrintManagerField pmField = injector.create(PrintManagerField.class);
        pmField.printDocument(); // Using Laser Printer

        PrintManagerCtor pmCtor = injector.create(PrintManagerCtor.class);
        pmCtor.printDocument(); // Using Inkjet Printer
    }
}
```

Reasoning to interviewer:

- @Qualifier lets me keep one interface with multiple beans and choose at the injection site, not deep inside the logic.
- Constructor injection is preferred—it makes the dependency graph explicit, immutable, and easy to test.
- A simple registry keeps the example focused. A production system would use ServiceLoader, classpath scanning, or a DI container (Spring/CDI).
- Supporting both field and parameter targets (@Target({FIELD, PARAMETER})) gives flexibility without extra annotations.

Future scope:

- Add scopes (@Singleton, @Prototype) and lifecycle hooks.
- Support named qualifiers for other interfaces (database clients, caches).
- Integrate with configuration (choose qualifier by environment).
- Add validation: fail fast if multiple candidates exist and no qualifier is present.

Scenario 16: Scheduling method execution with a custom annotation

You want a light, dependency-free way to run annotated methods at fixed intervals (e.g., health checks, cache warmers) without pulling in a scheduling framework.

Answer:

I'll declare @Scheduled(intervalMs=...) on parameterless methods. A small bootstrap scans the instance's methods, reads the interval, and submits repeating tasks to a ScheduledExecutorService. This avoids Timer's single-thread limitations and gives better error isolation. I'll also ensure methods are accessible, handle exceptions inside the task, and keep a reference to the scheduler to shut it down gracefully.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Scheduled {
    long intervalMs();
}

class TaskRunner {
    @Scheduled(intervalMs = 3000)
    public void recurringTask() {
        System.out.println("Recurring task at " + System.currentTimeMillis());
    }

    @Scheduled(intervalMs = 1500)
    private void internalPulse() {  // even private methods can be invoked reflectively
        System.out.println("Internal pulse at " + System.currentTimeMillis());
    }
}

class Scheduler implements AutoCloseable {
    private final ScheduledExecutorService pool =
Executors.newScheduledThreadPool(2);
    private final List<ScheduledFuture<?>> futures = new ArrayList<>();

    public void start(Object instance) {
        for (Method m : instance.getClass().getDeclaredMethods()) {
            Scheduled sched = m.getAnnotation(Scheduled.class);
```

```
            if (sched == null) continue;

            if (m.getParameterCount() != 0) {
                throw new IllegalArgumentException("@Scheduled methods must have
no parameters: " + m);
            }
            m.setAccessible(true);

            long interval = sched.intervalMs();
            Runnable task = () -> {
                try {
                    m.invoke(instance);
                } catch (Exception e) {
                    // swallow exceptions so the schedule keeps running; log in real
systems
                    System.err.println("Scheduled method failed: " + m.getName() + " -> "
+ e);
                }
            };

            // fixed-rate is typical for heartbeats; switch to fixed-delay if the work varies
a lot
            futures.add(pool.scheduleAtFixedRate(task, 0, interval,
TimeUnit.MILLISECONDS));
        }
    }

    @Override
    public void close() {
        futures.forEach(f -> f.cancel(false));
        pool.shutdown();
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        TaskRunner runner = new TaskRunner();
        try (Scheduler scheduler = new Scheduler()) {
            scheduler.start(runner);
            Thread.sleep(8000); // demo window
        }
        // scheduler closed -> tasks canceled, executor shut down
    }
}
```

Reasoning to interviewer:

- ScheduledExecutorService is safer than Timer: multiple threads, better exception isolation, and flexible policies (fixedRate vs fixedDelay).

• Restricting to parameterless methods keeps the contract simple and easier to invoke reflectively.
• Making methods accessible with setAccessible(true) allows private tasks without leaking them into the public API.
• A managed lifecycle (AutoCloseable) ensures clean shutdown—important for tests and services.

Future scope:

• Add attributes like initialDelayMs(), fixedDelay() vs fixedRate() mode, and cron() expressions.
• Support class-level enable/disable flags or environment profiles.
• Centralize error handling (retry/backoff) and structured logging.
• Allow concurrency controls (@Scheduled(concurrency=1)) to prevent re-entrancy for long-running tasks.

Scenario 17: Performance monitoring with annotations

You want to measure how long certain methods take without sprinkling timing code everywhere.

Answer:

I'll introduce a @MonitorPerformance annotation and a lightweight interceptor that wraps method execution. The wrapper will record start/end times, log duration, and optionally warn if a threshold is exceeded. I'll support methods with parameters and return values, and I'll keep failures visible by logging the exception and rethrowing it so behavior doesn't change.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.Arrays;
import java.util.concurrent.TimeUnit;

// 1) Annotation: optional threshold + unit for SLO-style alerts
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface MonitorPerformance {
    long threshold() default -1;              // -1 = disabled
    TimeUnit unit() default TimeUnit.MILLISECONDS;
}

// 2) Example service with a slow method
class Service {
    @MonitorPerformance(threshold = 200, unit = TimeUnit.MILLISECONDS)
    public String longRunningTask(String input) {
        try { Thread.sleep(250); } catch (InterruptedException ignored) {}
        return "Processed: " + input;
    }

    public int fastTask(int x) { return x * 2; } // not monitored
}

// 3) Interceptor utility that can invoke any method reflectively
class PerformanceMonitor {

    public static Object invoke(Object target, String methodName, Class<?>[]
paramTypes, Object... args) throws Throwable {
        Method m = target.getClass().getMethod(methodName, paramTypes);
        MonitorPerformance mp = m.getAnnotation(MonitorPerformance.class);
```

```
        if (mp == null) { // fall back to plain invoke if not annotated
            try { return m.invoke(target, args); }
            catch (InvocationTargetException ite) { throw ite.getTargetException(); }
        }

        long start = System.nanoTime();
        boolean success = false;
        try {
            Object result = m.invoke(target, args);
            success = true;
            return result;
        } catch (InvocationTargetException ite) {
            throw ite.getTargetException();
        } finally {
            long elapsedNs = System.nanoTime() - start;
            long elapsedInUnit = mp.unit().convert(elapsedNs,
TimeUnit.NANOSECONDS);

            String argStr = Arrays.toString(args);
            System.out.printf("[PERF] %s(%s) took %d %s %s%n",
                    m.getName(), argStr, elapsedInUnit, mp.unit().name(),
                    success ? "" : "(failed)");

            if (mp.threshold() >= 0 && elapsedInUnit > mp.threshold()) {
                System.out.printf("[PERF][WARN] %s exceeded threshold: %d %s > %d
%s%n",
                        m.getName(), elapsedInUnit, mp.unit().name(),
                        mp.threshold(), mp.unit().name());
            }
        }
    }
}

// 4) Demo
public class PerfMain {
    public static void main(String[] args) throws Throwable {
        Service svc = new Service();

        Object r1 = PerformanceMonitor.invoke(svc, "longRunningTask",
                new Class<?>[]{String.class}, "payload");
        System.out.println("Result: " + r1);

        Object r2 = PerformanceMonitor.invoke(svc, "fastTask",
                new Class<?>[]{int.class}, 21);
        System.out.println("Result: " + r2);
    }
}
```

Reasoning to interviewer:

- An annotation marks intent; the interceptor centralizes the cross-cutting timing concern.
- Using System.nanoTime() avoids wall-clock skew and gives stable measurements.
- I preserve method semantics: parameters/returns are handled, exceptions are rethrown.
- Threshold + unit let us set soft SLOs per method without hardcoding numbers in code.

Future scope:

- Export metrics to a registry (Micrometer/Prometheus) instead of printing.
- Add attributes like tags() to decorate metrics (service, operation).
- Use dynamic proxies or ByteBuddy to intercept interfaces/classes transparently without manual invoke(...) calls.
- Record percentiles and running averages; surface slowest N calls.

Scenario 18: Environment-based method access control via annotation

Some methods should only run in certain environments (e.g., “PRODUCTION” only).

Answer:

I’ll define an @EnvironmentRestriction annotation that accepts one or more allowed environments. Before invocation, an interceptor checks the current environment (e.g., from a system property). If the method isn’t allowed, it blocks with a clear exception. I’ll support both class-level and method-level annotations, with method-level taking precedence.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Method;
import java.util.EnumSet;

// 1) Strongly-typed environments via enum
enum Env { DEVELOPMENT, TEST, STAGING, PRODUCTION }

@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
@interface EnvironmentRestriction {
    Env[] value(); // allowed environments
}

// 2) Critical operations; class restricted to PROD, one method allowed in
STAGING too
@EnvironmentRestriction({Env.PRODUCTION})
class CriticalOperations {

    public void executeCriticalTask() {
        System.out.println("Critical operation performed");
    }

    @EnvironmentRestriction({Env.STAGING, Env.PRODUCTION})
    public void preProdDryRun() {
        System.out.println("Dry-run in staging/prod");
    }
}

// 3) Environment-aware invoker
class EnvValidator {
    // Read from external config; here simplified to system property with default
    static Env currentEnv() {
        String v = System.getProperty("app.env", "DEVELOPMENT").toUpperCase();
```

```
        return Env.valueOf(v);
    }

    public static Object validateAndInvoke(Object obj, Method method, Object...
args) throws Throwable {
        Env env = currentEnv();

        // Resolve effective restriction: method-level overrides class-level
        EnvironmentRestriction mAnno =
method.getAnnotation(EnvironmentRestriction.class);
        EnvironmentRestriction cAnno =
obj.getClass().getAnnotation(EnvironmentRestriction.class);

        EnumSet<Env> allowed = EnumSet.noneOf(Env.class);
        if (cAnno != null) allowed.addAll(EnumSet.of(cAnno.value()[0],
cAnno.value())); // add all
        if (mAnno != null) { // override if present
            allowed.clear();
            allowed.addAll(EnumSet.of(mAnno.value()[0], mAnno.value()));
        }

        if (!allowed.isEmpty() && !allowed.contains(env)) {
            throw new IllegalAccessException("Access to " + method.getName() + "
denied in " + env);
        }

        try {
            return method.invoke(obj, args);
        } catch (java.lang.reflect.InvocationTargetException ite) {
            throw ite.getTargetException();
        }
    }
}

// 4) Demo
public class EnvMain {
    public static void main(String[] args) throws Throwable {
        System.setProperty("app.env", "STAGING"); // try DEVELOPMENT /
STAGING / PRODUCTION

        CriticalOperations ops = new CriticalOperations();

        // Allowed in STAGING due to method-level override
        EnvValidator.validateAndInvoke(ops,
CriticalOperations.class.getMethod("preProdDryRun"));

        // Blocked in STAGING because class-level restriction is PRODUCTION-only
        try {
            EnvValidator.validateAndInvoke(ops,
CriticalOperations.class.getMethod("executeCriticalTask"));
```

```
        } catch (IllegalAccessException iae) {
            System.out.println("Blocked: " + iae.getMessage());
        }
    }
}
```

Reasoning to interviewer:

- Enum-based environments prevent typos and make restrictions explicit.
- Method-level annotation overrides class-level so we can narrowly open specific operations for non-prod.
- The guard fails fast with a precise error, preventing accidental execution in the wrong environment.
- The environment source is pluggable (system property, env var, config server).

Future scope:

- Add reason()/ticket() attributes to annotate why a method is restricted.
- Support deny lists (@Disallow({TEST, DEV})) or complex policies via predicates.
- Integrate with an AOP proxy so the checks happen transparently without manual validateAndInvoke.
- Emit audit logs/events when access is denied or allowed for traceability.

Scenario 19: Tracking how often methods are called using annotations

You want lightweight usage analytics on specific methods without sprinkling counters everywhere.

Answer:
I'll introduce a @TrackInvocation annotation and a tiny interceptor that increments a thread-safe counter each time an annotated method is invoked. I'll key counts by the full method signature (not just name) to avoid collisions across overloads/classes. For robustness, I'll unwrap InvocationTargetException so exceptions propagate correctly while still counting.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

// 1) Annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface TrackInvocation {}

// 2) Example service
class UsageService {
    @TrackInvocation
    public void processData() {
        System.out.println("Processing data");
    }

    @TrackInvocation
    public void processData(String tag) {
        System.out.println("Processing data for: " + tag);
    }

    public void helper() { /* not tracked */ }
}

// 3) Counter + interceptor
class InvocationCounter {

    // Key by full signature: declaring class + method name + parameter types
    private static final Map<String, AtomicLong> COUNTS = new
ConcurrentHashMap<>();
```

```
    private static String key(Method m) {
        StringBuilder sb = new StringBuilder(m.getDeclaringClass().getName())
                .append("#").append(m.getName()).append("(");
        Class<?>[] p = m.getParameterTypes();
        for (int i = 0; i < p.length; i++) {
            sb.append(p[i].getTypeName());
            if (i < p.length - 1) sb.append(",");
        }
        return sb.append(")").toString();
    }

    public static Object invokeWithCount(Object target, Method method, Object...
args) throws Throwable {
        if (method.isAnnotationPresent(TrackInvocation.class)) {
            String k = key(method);
            COUNTS.computeIfAbsent(k, __ -> new
AtomicLong()).incrementAndGet();
            System.out.println("[INVOC] " + k + " called " + COUNTS.get(k).get() + "
times");
        }
        try {
            return method.invoke(target, args);
        } catch (InvocationTargetException ite) {
            // propagate the original cause
            throw ite.getTargetException();
        }
    }

    public static long getCount(Method m) {
        return COUNTS.getOrDefault(key(m), new AtomicLong(0)).get();
    }
}

// 4) Demo
public class TrackMain {
    public static void main(String[] args) throws Throwable {
        UsageService svc = new UsageService();

        Method m1 = UsageService.class.getMethod("processData");
        Method m2 = UsageService.class.getMethod("processData", String.class);
        Method m3 = UsageService.class.getMethod("helper");

        InvocationCounter.invokeWithCount(svc, m1);
        InvocationCounter.invokeWithCount(svc, m1);
        InvocationCounter.invokeWithCount(svc, m2, "payments");
        // Not tracked
        InvocationCounter.invokeWithCount(svc, m3);
```

```
        System.out.println("processData() total = " +
InvocationCounter.getCount(m1));
        System.out.println("processData(String) total = " +
InvocationCounter.getCount(m2));
        System.out.println("helper() total = " + InvocationCounter.getCount(m3)); // 0
    }
}
```

Reasoning to interviewer:

- Using runtime retention + reflection allows us to increment counters only for annotated methods.
- A signature-based key prevents collisions between overloads or different classes.
- ConcurrentHashMap + AtomicLong make it safe and cheap in multi-threaded apps.
- We unwrap InvocationTargetException so failure semantics are unchanged.

Future scope:

- Export counts to a metrics system (Micrometer/Prometheus) and add tags like service name.
- Track additional dimensions (success/failure counts, latency buckets) next to invocations.
- Add a sampling rate in the annotation to reduce overhead on very hot methods.
- Provide an interface-based dynamic proxy so callers don’t need to use reflection explicitly.

Scenario 20: Circuit Breaker pattern implementation using annotations

You want to stop hammering an unreliable dependency: after N consecutive failures, open the breaker; after a cooldown, allow a probe; close on success.

Answer:

I'll design @CircuitBreaker(maxFailures, cooldownPeriodMs) and maintain a per-method state machine: CLOSED (normal), OPEN (reject calls until cooldown elapses), and HALF_OPEN (allow a limited test call). I'll store state in a concurrent map keyed by method signature. On each call:

- if OPEN and cooldown not elapsed → reject;
- if cooldown elapsed → transition to HALF_OPEN, let one call through;
- on success in HALF_OPEN → reset to CLOSED; on failure → go back to OPEN.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface CircuitBreaker {
    int maxFailures();
    long cooldownPeriodMs();
}

enum State { CLOSED, OPEN, HALF_OPEN }

class BreakerState {
    volatile State state = State.CLOSED;
    final AtomicInteger failures = new AtomicInteger(0);
    volatile long openedAtMs = 0L;
    volatile boolean halfOpenProbeInFlight = false;
}

class ExternalAPI {
```

```
    @CircuitBreaker(maxFailures = 3, cooldownPeriodMs = 3000)
    public void requestData() throws Exception {
        // simulate flakiness
        if (Math.random() < 0.6) throw new Exception("Remote service failure");
        System.out.println("Data retrieved successfully");
    }
}

class CircuitBreakerManager {

    private static final Map<String, BreakerState> STATES = new
ConcurrentHashMap<>();

    private static String key(Method m) {
        StringBuilder sb = new StringBuilder(m.getDeclaringClass().getName())
                .append("#").append(m.getName()).append("(");
        Class<?>[] p = m.getParameterTypes();
        for (int i = 0; i < p.length; i++) {
            sb.append(p[i].getTypeName());
            if (i < p.length - 1) sb.append(",");
        }
        return sb.append(")").toString();
    }

    public static Object execute(Object target, Method method, Object... args)
throws Throwable {
        CircuitBreaker cb = method.getAnnotation(CircuitBreaker.class);
        if (cb == null) {
            try { return method.invoke(target, args); }
            catch (InvocationTargetException ite) { throw ite.getTargetException(); }
        }

        String k = key(method);
        BreakerState s = STATES.computeIfAbsent(k, __ -> new BreakerState());
        long now = System.currentTimeMillis();

        // State gate
        if (s.state == State.OPEN) {
            long elapsed = now - s.openedAtMs;
            if (elapsed < cb.cooldownPeriodMs()) {
                throw new IllegalStateException("Circuit OPEN for " + k + ", wait " +
(cb.cooldownPeriodMs()-elapsed) + "ms");
            }
            // Try half-open probe
            s.state = State.HALF_OPEN;
            s.halfOpenProbeInFlight = true;
        } else if (s.state == State.HALF_OPEN) {
            // allow only one probe at a time
            if (s.halfOpenProbeInFlight) {
```

```
                throw new IllegalStateException("Circuit HALF_OPEN probe in flight for
" + k);
            }
            s.halfOpenProbeInFlight = true;
        }

        try {
            Object result = method.invoke(target, args);

            // Success path
            s.failures.set(0);
            if (s.state == State.HALF_OPEN) {
                s.state = State.CLOSED;
                s.halfOpenProbeInFlight = false;
                System.out.println("[CB] " + k + " CLOSED (recovered)");
            }
            return result;

        } catch (InvocationTargetException ite) {
            Throwable cause = ite.getTargetException();

            int f = s.failures.incrementAndGet();
            if (s.state == State.HALF_OPEN) {
                // fail fast back to OPEN
                s.state = State.OPEN;
                s.openedAtMs = now;
                s.halfOpenProbeInFlight = false;
                System.out.println("[CB] " + k + " back to OPEN (half-open failed)");
            } else if (s.state == State.CLOSED && f >= cb.maxFailures()) {
                s.state = State.OPEN;
                s.openedAtMs = now;
                System.out.println("[CB] " + k + " OPEN (failures=" + f + ")");
            }
            throw cause; // propagate business exception
        } finally {
            // Clear probe flag if we were the probe
            if (s.state == State.HALF_OPEN && s.halfOpenProbeInFlight) {
                // leave flag true so other threads don't enter until we finish;
                // it gets cleared on success path above or when we switch to OPEN on
failure.
            }
        }
    }
}

// Demo
public class CbMain {
    public static void main(String[] args) throws Exception {
        ExternalAPI api = new ExternalAPI();
        Method call = ExternalAPI.class.getMethod("requestData");
```

```
        for (int i = 0; i < 12; i++) {
            try {
                CircuitBreakerManager.execute(api, call);
            } catch (Throwable t) {
                System.out.println("Call " + i + " -> " + t.getMessage());
            }
            Thread.sleep(400); // simulate client cadence
        }
    }
}
```

Reasoning to interviewer:

- The annotation declares policy; the manager enforces it via a per-method state machine.
- We key state by full signature to avoid clashes across overloads.
- CLOSED → OPEN after maxFailures; OPEN → HALF_OPEN after cooldown; success in HALF_OPEN closes the breaker, failure re-opens it.
- We limit the half-open probe to a single in-flight call to prevent dog-piling an unstable service.

Future scope:

- Add parameters like windowMs (failure window), halfOpenMaxCalls, and fallbackMethod() to degrade gracefully.
- Emit metrics (open/close events, rejection counts) and integrate with logging/alerts.
- Use an AOP proxy (dynamic proxies/ByteBuddy/Spring AOP) to apply automatically without manual reflection calls.
- Persist breaker state if you need continuity across process restarts.

Scenario 21: Automating method entry and exit logging using annotations

You want clean business code with consistent logs every time a method starts and finishes.

Answer:
I'll introduce a @LogExecution annotation and a small interceptor that surrounds method calls with "enter" and "exit" logs. I'll include arguments and elapsed time so the log line is actually useful during debugging. Exceptions will be logged and rethrown so behavior doesn't change. This is the same idea AOP frameworks use; here we do it with plain reflection for clarity.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.Arrays;

// 1) Annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface LogExecution {
    String value() default ""; // optional label
}

// 2) Service with an annotated method
class OrderService {
    @LogExecution("order-flow")
    public String placeOrder(String sku, int qty) {
        System.out.println("Order placement in progress for " + sku + " x" + qty);
        return "OK";
    }

    public void helper() {} // not logged
}

// 3) Interceptor that wraps calls with entry/exit logs + timing
class ExecutionLogger {

    public static Object invokeWithLogging(Object instance, Method method,
Object... args) throws Throwable {
        LogExecution meta = method.getAnnotation(LogExecution.class);
        if (meta == null) {
            try { return method.invoke(instance, args); }
```

```
            catch (InvocationTargetException ite) { throw ite.getTargetException(); }
        }

        String label = meta.value().isEmpty() ? method.getName() : meta.value();
        String argStr = Arrays.toString(args);

        long start = System.nanoTime();
        System.out.printf("[ENTER] %s %s args=%s%n",
instance.getClass().getSimpleName()+"#"+method.getName(), label, argStr);

        try {
            Object result = method.invoke(instance, args);
            long tookMs = (System.nanoTime() - start) / 1_000_000;
            System.out.printf("[EXIT ] %s %s took=%dms result=%s%n",
instance.getClass().getSimpleName()+"#"+method.getName(), label, tookMs,
result);
            return result;
        } catch (InvocationTargetException ite) {
            long tookMs = (System.nanoTime() - start) / 1_000_000;
            Throwable cause = ite.getTargetException();
            System.out.printf("[ERROR] %s %s took=%dms ex=%s%n",
instance.getClass().getSimpleName()+"#"+method.getName(), label, tookMs,
cause);
            throw cause;
        }
    }
}

// 4) Demo runner: invoke all @LogExecution methods
public class Runner {
    public static void main(String[] args) throws Throwable {
        OrderService service = new OrderService();
        for (Method m : service.getClass().getDeclaredMethods()) {
            if (m.isAnnotationPresent(LogExecution.class)) {
                ExecutionLogger.invokeWithLogging(service, m, "SKU-123", 2);
            }
        }
    }
}
```

Reasoning to interviewer:

- An annotation declares intent; a centralized interceptor keeps logs consistent and avoids cut-and-paste System.out.println everywhere.
- Using System.nanoTime() gives stable timing and we keep the method's signature/args in the log for real troubleshooting.

- We unwrap InvocationTargetException so the original exception propagates, preserving semantics.

Future scope:

- Add attributes like level() (INFO/DEBUG) and logArgs() to control verbosity.
- Integrate with SLF4J/Log4j and MDC to include correlation IDs or request context.
- Apply transparently via dynamic proxies/ByteBuddy/Spring AOP so callers don’t touch reflection.

Scenario 22: Automatic exception logging with custom annotations

You want annotated methods to auto-log failures without littering code with try–catch blocks.

Answer:

I'll add a @LogException annotation. The invocation handler will wrap the call, catch InvocationTargetException, log the root cause (type, message, maybe stack trace), and then rethrow it. This separates cross-cutting error logging from business logic. I'll also support a rethrow() option in case you want to swallow certain errors later.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;

// 1) Annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface LogException {
    boolean rethrow() default true;   // keep default behavior: rethrow after logging
}

// 2) Business code with an annotated method that can fail
class PaymentProcessor {
    @LogException
    public void processPayment(String userId, double amount) throws Exception {
        throw new Exception("Insufficient funds for " + userId + " amount=" +
amount);
    }

    public void ping() { /* not logged */ }
}

// 3) Exception-logging invoker
class ExceptionHandler {

    public static Object invokeWithExceptionLogging(Object obj, Method method,
Object... args) throws Throwable {
        LogException le = method.getAnnotation(LogException.class);
        if (le == null) {
            try { return method.invoke(obj, args); }
            catch (InvocationTargetException ite) { throw ite.getTargetException(); }
        }

        try {
```

```
            return method.invoke(obj, args);
        } catch (InvocationTargetException ite) {
            Throwable cause = ite.getTargetException();
            // Log key details; in real apps, use a logger and include stack trace
            System.out.printf("[EXC ] %s#%s failed: %s: %s%n",
                    obj.getClass().getSimpleName(),
                    method.getName(),
                    cause.getClass().getSimpleName(),
                    cause.getMessage());

            if (le.rethrow()) throw cause; // preserve behavior
            return null; // swallow if configured (not typical, but supported)
        }
    }
}

// 4) Demo
public class TestApp {
    public static void main(String[] args) throws Throwable {
        PaymentProcessor processor = new PaymentProcessor();
        for (Method m : processor.getClass().getDeclaredMethods()) {
            if (m.isAnnotationPresent(LogException.class)) {
                try {
                    ExceptionHandler.invokeWithExceptionLogging(processor, m, "u-42",
199.0);
                } catch (Throwable ignored) {}
            }
        }
    }
}
```

Reasoning to interviewer:

- The annotation flips the code from imperative try–catch to declarative error logging.
- We always log the root cause (getTargetException()), not the wrapper.
- A rethrow flag gives policy control; defaulting to true avoids masking errors.

Future scope:

- Include logStackTrace() and category() attributes for richer logging controls.
- Emit structured logs (JSON) with fields like user, orderId, correlationId from MDC.
- Add retry policies or fallback methods (e.g., @Fallback(method="alt")) to degrade gracefully.

- Wire it in via proxies/AOP so all calls through an interface are automatically intercepted.

Scenario 23: Adding retry mechanism via annotations

Transient failures (network blips, timeouts) should be retried automatically without cluttering business code.

Answer:

I'll define a @Retry annotation with knobs like maxRetries, backoffMs, and jitterMs. A small interceptor will detect the annotation, execute the method, and on failure retry up to the limit with exponential backoff + jitter. I'll unwrap InvocationTargetException so the original cause is preserved, and allow an exception filter to retry only on specific exception types.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.concurrent.ThreadLocalRandom;

// 1) Annotation with sensible defaults
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Retry {
    int maxRetries() default 3;           // total attempts = maxRetries
    long backoffMs() default 200;         // base backoff
    long jitterMs() default 100;          // add randomness to avoid thundering herds
    Class<? extends Throwable>[] retryOn() default { Exception.class }; // whitelist
}

// 2) Example business code
class NetworkClient {
    @Retry(maxRetries = 4, backoffMs = 250, jitterMs = 150, retryOn = {
java.io.IOException.class })
    public void establishConnection() throws Exception {
        if (Math.random() < 0.75) {
            throw new java.io.IOException("Failed to connect (transient)");
        }
        System.out.println("Connection successful");
    }
}

// 3) Interceptor with exponential backoff + jitter and exception filtering
class RetryExecutor {

    public static Object runWithRetries(Object target, Method method, Object...
args) throws Throwable {
        Retry cfg = method.getAnnotation(Retry.class);
```

```
        if (cfg == null) { // not annotated
            try { return method.invoke(target, args); }
            catch (InvocationTargetException ite) { throw ite.getTargetException(); }
        }

        int attempts = cfg.maxRetries();
        long base = cfg.backoffMs();
        long jitter = cfg.jitterMs();

        for (int attempt = 1; attempt <= attempts; attempt++) {
            try {
                return method.invoke(target, args);
            } catch (InvocationTargetException ite) {
                Throwable cause = ite.getTargetException();
                if (!isRetryable(cause, cfg.retryOn()) || attempt == attempts) {
                    // Not retryable OR out of attempts -> propagate
                    throw cause;
                }
                long sleep = backoffWithJitter(base, attempt, jitter);
                System.out.printf("[RETRY] %s attempt %d/%d failed: %s. Sleeping
%dms%n",
                        method.getName(), attempt, attempts, cause.getMessage(), sleep);
                sleepQuietly(sleep);
            }
        }
        return null; // unreachable
    }

    private static boolean isRetryable(Throwable t, Class<? extends Throwable>[]
whitelist) {
        for (Class<? extends Throwable> c : whitelist) {
            if (c.isInstance(t)) return true;
        }
        return false;
    }

    private static long backoffWithJitter(long base, int attempt, long jitter) {
        long exp = base * (1L << (attempt - 1)); // exponential: base, 2x, 4x...
        long rand = ThreadLocalRandom.current().nextLong(0, Math.max(1, jitter));
        return exp + rand;
    }

    private static void sleepQuietly(long ms) {
        try { Thread.sleep(ms); } catch (InterruptedException ignored) {
Thread.currentThread().interrupt(); }
    }
}

// 4) Demo
public class ApplicationRunner {
```

```
    public static void main(String[] args) throws Throwable {
        NetworkClient client = new NetworkClient();
        Method m = NetworkClient.class.getMethod("establishConnection");
        RetryExecutor.runWithRetries(client, m);
    }
}
```

Reasoning to interviewer:

- The annotation declares policy; the interceptor implements it uniformly.
- Exponential backoff with jitter reduces load spikes and contention on recovering services.
- retryOn whitelists transient exceptions, so we don’t retry on, say, IllegalArgumentException.
- Exceptions are unwrapped and rethrown to preserve semantics; the interceptor is transparent beyond retries.

Future scope:

- Add maxDurationMs (overall cap), timeoutMs per attempt, and circuit-breaker integration.
- Track metrics (retries, last error) and expose them via Micrometer.
- Support async methods with CompletableFuture by decorating with retry logic.
- Allow @Fallback(method="...") for graceful degradation on final failure.

Scenario 24: Controlling method call frequency with a rate limiting annotation

Some endpoints need throttling to protect downstream systems and keep QoS stable.

Answer:
I'll provide a @RateLimit(minIntervalMs=...) annotation and an enforcer that tracks the last-invocation timestamp per method signature. Before invoking, it checks the delta and rejects calls that arrive too soon. For better multi-thread safety and robustness, I'll use a ConcurrentHashMap keyed by full signature and update timestamps atomically. I'll also show an optional token-bucket variant if burst tolerance is needed.

Code (simple min-interval limiter):

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

// 1) Annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface RateLimit {
    long minIntervalMs();
}

// 2) Service
class DataService {
    @RateLimit(minIntervalMs = 1500)
    public void retrieveData() {
        System.out.println("Retrieving data @ " + System.currentTimeMillis());
    }
}

// 3) Enforcer keyed by full method signature
class RateLimitEnforcer {
    private static final Map<String, Long> lastCall = new ConcurrentHashMap<>();

    public static Object invokeWithRateControl(Object instance, Method method,
Object... args) throws Throwable {
        RateLimit cfg = method.getAnnotation(RateLimit.class);
        if (cfg == null) {
            try { return method.invoke(instance, args); }
            catch (InvocationTargetException ite) { throw ite.getTargetException(); }
        }
```

```
        String key = signature(method);
        long now = System.currentTimeMillis();
        long prev = lastCall.getOrDefault(key, 0L);
        long delta = now - prev;

        if (delta < cfg.minIntervalMs()) {
            throw new IllegalStateException("Rate limit exceeded for " + key + " (wait "
+ (cfg.minIntervalMs() - delta) + "ms)");
        }

        // record invocation time (best-effort; minor race is acceptable here)
        lastCall.put(key, now);

        try {
            return method.invoke(instance, args);
        } catch (InvocationTargetException ite) {
            throw ite.getTargetException();
        }
    }

    private static String signature(Method m) {
        StringBuilder sb = new StringBuilder(m.getDeclaringClass().getName())
                .append("#").append(m.getName()).append("(");
        Class<?>[] p = m.getParameterTypes();
        for (int i = 0; i < p.length; i++) {
            sb.append(p[i].getTypeName());
            if (i < p.length - 1) sb.append(",");
        }
        return sb.append(")").toString();
    }
}

// 4) Demo
public class RateLimitApp {
    public static void main(String[] args) throws Throwable {
        DataService svc = new DataService();
        Method m = DataService.class.getMethod("retrieveData");

        for (int i = 0; i < 4; i++) {
            try {
                RateLimitEnforcer.invokeWithRateControl(svc, m);
            } catch (Throwable t) {
                System.out.println(t.getMessage());
            }
            Thread.sleep(700); // observe throttling
        }
    }
}
```

Reasoning to interviewer:

- Per-method signature keys avoid collisions across overloads/classes.
- A simple “min interval” gate is cheap and effective for low-traffic admin ops or cron-like tasks.
- Using ConcurrentHashMap keeps it thread-safe; we accept tiny races since the policy is best-effort.

Future scope:

- Token-bucket or leaky-bucket algorithms for burst tolerance and steady average rate (e.g., N calls per second).
- Per-tenant / per-user keys by combining method signature with caller id for fair usage control.
- Async backpressure: queue or delay rather than throwing, depending on SLA.
- Metrics and logs (rejections, current rate) and dynamic config reload for live tuning.

Scenario 25: Measuring method execution time for performance analysis

You want timing around selected methods without littering business code with start/stop calls.

Answer:

I'll add a @Timed annotation and a tiny wrapper that measures duration with System.nanoTime() (higher resolution and not wall-clock skewed). The wrapper logs args, result (optionally), and the time taken. If the method throws, we still log the time and rethrow the original exception so behavior is unchanged.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.Arrays;
import java.util.concurrent.TimeUnit;

// 1) Annotation with a couple of knobs
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Timed {
    TimeUnit unit() default TimeUnit.MILLISECONDS;
    boolean logArgs() default true;
    boolean logResult() default false;
}

// 2) Example service
class TaskProcessor {
    @Timed(unit = TimeUnit.MILLISECONDS, logArgs = true, logResult = false)
    public void executeHeavyTask(String jobId) throws InterruptedException {
        Thread.sleep(1200); // simulate heavy work
        System.out.println("Heavy task finished for " + jobId);
    }

    public void helper() { /* not timed */ }
}

// 3) Timing utility that keeps semantics intact
class TimingUtility {
    public static Object invokeWithTiming(Object target, Method method, Object...
args) throws Throwable {
        Timed timed = method.getAnnotation(Timed.class);
        if (timed == null) {
            try { return method.invoke(target, args); }
```

```
            catch (InvocationTargetException ite) { throw ite.getTargetException(); }
        }

        String sig = target.getClass().getSimpleName() + "#" + method.getName();
        String argStr = timed.logArgs() ? Arrays.toString(args) : "[hidden]";
        long start = System.nanoTime();

        try {
            Object result = method.invoke(target, args);
            long tookNs = System.nanoTime() - start;
            long took = timed.unit().convert(tookNs, TimeUnit.NANOSECONDS);
            System.out.printf("[TIMED] %s args=%s took=%d %s%s%n",
                    sig, argStr, took, timed.unit().name(),
                    timed.logResult() ? (" result=" + result) : "");
            return result;
        } catch (InvocationTargetException ite) {
            long tookNs = System.nanoTime() - start;
            long took = timed.unit().convert(tookNs, TimeUnit.NANOSECONDS);
            Throwable cause = ite.getTargetException();
            System.out.printf("[TIMED][ERROR] %s args=%s took=%d %s ex=%s%n",
                    sig, argStr, took, timed.unit().name(), cause);
            throw cause;
        }
    }
}

// 4) Demo
public class PerformanceMonitorApp {
    public static void main(String[] args) throws Throwable {
        TaskProcessor processor = new TaskProcessor();

        for (Method m : processor.getClass().getDeclaredMethods()) {
            if (m.isAnnotationPresent(Timed.class)) {
                TimingUtility.invokeWithTiming(processor, m, "JOB-987");
            }
        }
    }
}
```

Reasoning to interviewer:

- @Timed is a declarative switch; the timing concern lives in one place, not scattered.
- System.nanoTime() gives consistent elapsed timing; we convert to the requested unit.
- We log even on exceptions, then rethrow the cause to preserve method semantics.

- Optional logging of args/result helps during diagnostics but can be turned off for noisy or sensitive methods.

Future scope:

- Export timings to a metrics backend (Micrometer/Prometheus), compute percentiles (p95/p99).
- Add threshold() to flag slow calls and emit warnings.
- Intercept via dynamic proxies/ByteBuddy/Spring AOP so callers don’t write reflection code.
- Redact sensitive arguments with a companion parameter annotation like @Sensitive.

Scenario 26: Enforcing preconditions using custom annotations

Certain methods should run only if inputs satisfy constraints (e.g., “number must be ≥ 20”), and you want that policy expressed declaratively.

Answer:
I’ll create a @Requires annotation that declares a minimum threshold and which parameter to check. The validator inspects the annotation, evaluates the precondition against the chosen argument, and throws a clear exception if it fails. This separates guard logic from business logic and keeps rules visible at the method signature.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.lang.reflect.Parameter;

// 1) Annotation: choose which parameter to validate and a minimum value
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Requires {
    int paramIndex() default 0;  // which arg to check
    int minThreshold();
    String message() default ""; // custom error message
}

// 2) Business class
class MathOperations {

    // Require the first parameter to be >= 20
    @Requires(minThreshold = 20, paramIndex = 0, message = "Input too small for
safe calculation")
    public int performCalculation(int number) {
        System.out.println("Calculating with: " + number);
        return number * 2;
    }

    // Multiple params: validate second parameter
    @Requires(minThreshold = 10, paramIndex = 1)
    public int addWithLowerBound(int a, int b) {
        return a + b;
    }
}

// 3) Precondition validator
class PreconditionValidator {
```

```
    public static Object validateAndExecute(Object obj, Method method, Object...
args) throws Throwable {
        Requires req = method.getAnnotation(Requires.class);
        if (req == null) {
            try { return method.invoke(obj, args); }
            catch (InvocationTargetException ite) { throw ite.getTargetException(); }
        }

        int idx = req.paramIndex();
        if (idx < 0 || idx >= args.length) {
            throw new IllegalArgumentException("@Requires paramIndex out of
bounds for " + method);
        }

        Object arg = args[idx];
        if (!(arg instanceof Number)) {
            throw new IllegalArgumentException("@Requires can only validate
numeric arguments; got " + arg);
        }

        int value = ((Number) arg).intValue();
        if (value < req.minThreshold()) {
            String defaultMsg = "Precondition failed: arg[" + idx + "] = " + value +
                                " < minThreshold " + req.minThreshold();
            String msg = req.message().isEmpty() ? defaultMsg : req.message() + "
(got " + value + ")";
            throw new IllegalArgumentException(msg);
        }

        try {
            return method.invoke(obj, args);
        } catch (InvocationTargetException ite) {
            throw ite.getTargetException();
        }
    }
}

// 4) Demo
public class PreconditionsApp {
    public static void main(String[] args) throws Throwable {
        MathOperations ops = new MathOperations();

        Method m1 = MathOperations.class.getMethod("performCalculation",
int.class);
        Method m2 = MathOperations.class.getMethod("addWithLowerBound",
int.class, int.class);

        // Will fail
        try {
            PreconditionValidator.validateAndExecute(ops, m1, 10);
```

```
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }

        // Will pass
        Object r1 = PreconditionValidator.validateAndExecute(ops, m1, 25);
        System.out.println("Result: " + r1);

        // Validate second parameter (index 1)
        try {
            PreconditionValidator.validateAndExecute(ops, m2, 5, 7); // passes (b=7
>= 10? no -> will fail)
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        Object r2 = PreconditionValidator.validateAndExecute(ops, m2, 5, 12); // now
OK
        System.out.println("Result: " + r2);
    }
}
```

Reasoning to interviewer:

- The annotation is declarative and local to the method, making the rule obvious during reviews.
- paramIndex avoids guesswork about which argument is constrained; we check exactly that parameter.
- We fail fast with a clear message and keep business logic uncluttered.
- Reflection unpacks and rethrows the original exceptions, preserving method behavior.

Future scope:

- Add more predicates (e.g., max(), notBlank()), or support multiple constraints by allowing repeatable annotations.
- Parameter-level constraints (@Min, @Max) with validation at invocation time (similar to Bean Validation/JSR-380).
- Compose with @Timed/@LogException for a stacked, annotation-driven policy.
- Integrate with a validation framework to build rich rules (cross-parameter checks, custom validators).

Scenario 27: Automating exception handling using annotations

You want risky methods to automatically log failures and optionally alert admins—without littering the code with try–catch blocks.

Answer:

I'll define @HandleException with attributes like alert() (whether to notify ops) and message() (context). An interceptor wraps calls to annotated methods, catches the original exception (unwrapping InvocationTargetException), logs details (type, message), triggers alerts if requested, and then either rethrows or swallows based on policy. This keeps cross-cutting error handling centralized and consistent.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;

// 1) Annotation with knobs for behavior
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface HandleException {
    boolean alert() default true;         // send alert when failures occur
    boolean rethrow() default true;       // rethrow after handling
    String message() default "";          // extra context in logs/alerts
}

// 2) Business code
class OperationService {
    @HandleException(message = "Risky operation failed", alert = true, rethrow =
false)
    public void performRiskyTask() throws Exception {
        throw new Exception("Unexpected failure occurred");
    }

    @HandleException(rethrow = true)
    public void mustPropagate() throws Exception {
        throw new IllegalStateException("Must be visible to caller");
    }
}

// 3) Exception processor
class ExceptionProcessor {

    public static Object executeWithExceptionHandling(Object target, Method
method, Object... args) throws Throwable {
        HandleException meta = method.getAnnotation(HandleException.class);
```

```
        if (meta == null) {
            try { return method.invoke(target, args); }
            catch (InvocationTargetException ite) { throw ite.getTargetException(); }
        }

        try {
            return method.invoke(target, args);
        } catch (InvocationTargetException ite) {
            Throwable cause = ite.getTargetException();

            // Log structured details; replace with a real logger in production
            String msg = meta.message().isEmpty() ? method.getName() :
meta.message();
            System.out.printf("[ERROR] %s: %s: %s%n",
                    msg, cause.getClass().getSimpleName(), cause.getMessage());

            // Optional alerting hook (email/SMS/Slack page through your notifier)
            if (meta.alert()) {
                Alerting.notify("Exception in " + method.getName(), cause);
            }

            if (meta.rethrow()) throw cause; // preserve caller semantics
            return null; // swallowed by policy
        }
    }
}

// 4) Simplified alerting hook (placeholder)
class Alerting {
    public static void notify(String title, Throwable t) {
        System.out.println("[ALERT] " + title + " :: " + t);
    }
}

// 5) Demo
public class AppRunner {
    public static void main(String[] args) throws Throwable {
        OperationService svc = new OperationService();

        for (Method m : svc.getClass().getDeclaredMethods()) {
            if (m.isAnnotationPresent(HandleException.class)) {
                try {
                    ExceptionProcessor.executeWithExceptionHandling(svc, m);
                } catch (Throwable t) {
                    System.out.println("Propagated: " + t.getMessage());
                }
            }
        }
    }
}
```

Reasoning to interviewer:

- The annotation declares policy; the processor enforces it consistently across methods.
- We unwrap InvocationTargetException to get the real cause and log that—otherwise logs are noisy and misleading.
- A simple alerting hook demonstrates where integrations (email/Slack/PagerDuty) plug in.
- rethrow lets us choose between fail-fast and “best effort” flows depending on the operation’s criticality.

Future scope:

- Add categories() or severity() to route alerts to different channels.
- Capture stack traces and correlation IDs (MDC) for better incident triage.
- Count errors and emit metrics; open a circuit breaker if error rate spikes.
- Apply transparently via dynamic proxies/AOP so callers don’t need to use reflection manually.

Scenario 28: Implementing dependency injection via custom annotations

You want to remove manual new calls and wire dependencies automatically, keeping classes modular and testable.

Answer:

I'll create an @Inject field/parameter annotation and a minimal container that:

1. discovers injectable points (fields or constructor params),
2. constructs required dependencies (using a registry to honor named qualifiers/scopes), and
3. sets them via reflection. I'll prefer constructor injection (immutable, test-friendly), but also support field injection for brevity.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.HashMap;
import java.util.Map;

// 1) DI annotations
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.FIELD, ElementType.PARAMETER})
@interface Inject {}

@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.FIELD, ElementType.PARAMETER})
@interface Named {
    String value();
}

// 2) Components
interface Motor { String kind(); }

class ElectricMotor implements Motor { public String kind() { return "Electric"; } }
class DieselMotor   implements Motor { public String kind() { return "Diesel"; } }

// 3) Example consumer with constructor injection (preferred) + a field fallback
class Vehicle {
    private final Motor primary;

    @Inject @Named("backup")
    private Motor backup; // field injection for secondary dependency

    // Constructor DI: pick the "greediest" constructor and satisfy @Inject params
    public Vehicle(@Inject @Named("primary") Motor primary) {
```

```
        this.primary = primary;
    }

    public void run() {
        System.out.println("Primary motor: " + primary.kind());
        System.out.println("Backup motor: " + (backup != null ? backup.kind() :
"none"));
    }
}

// 4) Tiny container with a registry
class Container {
    private final Map<String, Object> singletons = new HashMap<>();

    public Container() {
        // register named beans
        singletons.put("primary", new ElectricMotor());
        singletons.put("backup",  new DieselMotor());
    }

    // Create and inject an object of the given type
    public <T> T create(Class<T> type) {
        try {
            // Choose constructor: prefer one with @Inject params (or the longest)
            Constructor<?> chosen = null;
            for (Constructor<?> c : type.getDeclaredConstructors()) {
                if (chosen == null || c.getParameterCount() >
chosen.getParameterCount()) chosen = c;
            }

            T instance;
            if (chosen == null || chosen.getParameterCount() == 0) {
                instance = type.getDeclaredConstructor().newInstance();
            } else {
                Object[] args = resolveParameters(chosen);
                @SuppressWarnings("unchecked")
                T tmp = (T) chosen.newInstance(args);
                instance = tmp;
            }

            injectFields(instance);
            return instance;
        } catch (Exception e) {
            throw new RuntimeException("DI failed for " + type.getName(), e);
        }
    }

    private Object[] resolveParameters(Constructor<?> ctor) {
        Parameter[] params = ctor.getParameters();
        Object[] args = new Object[params.length];
```

```
        for (int i = 0; i < params.length; i++) {
            Parameter p = params[i];
            if (!p.isAnnotationPresent(Inject.class)) {
                throw new IllegalStateException("All constructor params must be
@Inject: " + ctor);
            }
            String name = p.isAnnotationPresent(Named.class) ?
p.getAnnotation(Named.class).value() : null;
            args[i] = resolveByTypeAndName(p.getType(), name);
        }
        return args;
    }

    private void injectFields(Object target) throws IllegalAccessException {
        for (Field f : target.getClass().getDeclaredFields()) {
            if (!f.isAnnotationPresent(Inject.class)) continue;
            String name = f.isAnnotationPresent(Named.class) ?
f.getAnnotation(Named.class).value() : null;
            Object dep = resolveByTypeAndName(f.getType(), name);
            f.setAccessible(true);
            f.set(target, dep);
        }
    }

    private Object resolveByTypeAndName(Class<?> type, String name) {
        if (name != null) {
            Object bean = singletons.get(name);
            if (bean == null || !type.isInstance(bean)) {
                throw new IllegalStateException("No bean named '" + name + "' for type
" + type.getName());
            }
            return bean;
        }
        // simple fallback: find any matching singleton by type
        return singletons.values().stream()
                .filter(type::isInstance)
                .findFirst()
                .orElseThrow(() -> new IllegalStateException("No bean for type " +
type.getName()));
    }
}

// 5) Demo
public class DependencyInjectionDemo {
    public static void main(String[] args) {
        Container container = new Container();
        Vehicle v = container.create(Vehicle.class);
        v.run();
    }
}
```

Reasoning to interviewer:

- Constructor injection makes dependencies explicit and supports immutability; field injection is convenient for optional/secondary components.
- @Named disambiguates multiple implementations of the same interface (akin to qualifiers).
- A registry keeps the example simple; a real container would do classpath scanning, scopes, and lifecycle hooks.
- Reflection-based wiring centralizes object creation, reducing coupling and boilerplate.

Future scope:

- Add scopes (@Singleton, @Prototype) and post-construct hooks.
- Detect circular dependencies and fail early with a clear diagnostic.
- Integrate with configuration to choose implementations per environment.
- Replace manual reflection with dynamic proxies/AOP to add cross-cutting concerns (logging, metrics) on injected beans.

Scenario 29: Building a custom validation framework using annotations

You want constraints like “username must not be empty” and “password must be at least 6 chars” to be declared near the fields, while a single validator enforces them uniformly.

Answer:

I’ll define lightweight annotations (@NotEmpty, @MinLength) with runtime retention. The validator will reflect over fields, check which constraints apply, and collect violations instead of failing fast—so the caller gets a complete error report. I’ll also handle null values safely, support private fields, and return a structured list of errors (or throw a single exception carrying them). This keeps validation logic centralized and out of your business code.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

// ====== Annotations ======
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotEmpty {
    String message() default "must not be empty";
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MinLength {
    int value();
    String message() default "length is too short";
}

// ====== Model ======
class Account {
    @NotEmpty(message = "username is required")
    private String username;

    @MinLength(value = 6, message = "password must be at least 6 chars")
    private String password;

    // setters for demo
    public void setUsername(String username) { this.username = username; }
```

```
    public void setPassword(String password) { this.password = password; }
}

// ====== Validation Result ======
class Violation {
    final String field;
    final String message;

    Violation(String field, String message) {
        this.field = field;
        this.message = message;
    }

    @Override public String toString() { return field + ": " + message; }
}

class ValidationException extends RuntimeException {
    final List<Violation> violations;
    ValidationException(List<Violation> v) {
        super("Validation failed with " + v.size() + " error(s)");
        this.violations = v;
    }
    public List<Violation> getViolations() { return violations; }
}

// ====== Validator ======
class ValidationProcessor {

    public static void validate(Object bean) {
        List<Violation> violations = new ArrayList<>();

        for (Field f : bean.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            Object value;
            try {
                value = f.get(bean);
            } catch (IllegalAccessException e) {
                violations.add(new Violation(f.getName(), "unreadable field"));
                continue;
            }

            // @NotEmpty: supports String only in this simple demo
            NotEmpty notEmpty = f.getAnnotation(NotEmpty.class);
            if (notEmpty != null) {
                if (!(value instanceof String) || ((String) value).trim().isEmpty()) {
                    violations.add(new Violation(f.getName(), notEmpty.message()));
                }
            }

            // @MinLength: supports String only in this simple demo
```

```
            MinLength minLength = f.getAnnotation(MinLength.class);
            if (minLength != null) {
                String s = (value == null) ? null : value.toString();
                if (s == null || s.length() < minLength.value()) {
                    violations.add(new Violation(
                        f.getName(),
                        minLength.message() + " (min=" + minLength.value() + ")"
                    ));
                }
            }
        }

        if (!violations.isEmpty()) throw new ValidationException(violations);
    }
}

// ====== Demo ======
public class ValidatorDemo {
    public static void main(String[] args) {
        Account account = new Account();
        account.setUsername("Alice");
        account.setPassword("123"); // too short

        try {
            ValidationProcessor.validate(account);
            System.out.println("Valid!");
        } catch (ValidationException ve) {
            System.out.println(ve.getMessage());
            ve.getViolations().forEach(v -> System.out.println(" - " + v));
        }
    }
}
```

Reasoning to interviewer:

- Annotations co-locate rules with the data model; the validator centralizes enforcement.
- I collect all errors before throwing so the UI can show a complete list in one pass.
- Reflection with setAccessible(true) lets me validate private fields without changing the domain classes.
- The design is extendable: every new constraint is just a new annotation plus a small check.

Future scope:

- Add more constraints (@Email, @Range, @Pattern) and make them repeatable.
- Support nested objects (walk the graph) and collections (validate each element).
- Plug into Bean Validation (JSR-380) style with javax.validation.Constraint if you need standardization.
- Provide a Validator interface and chain multiple validators (domain-specific rules, cross-field checks).

Scenario 30: Adding logging via dynamic proxy

You want to log interface method calls (name, args, timing) without touching the target class.

Answer:

Dynamic proxies let me create a runtime wrapper around any interface. I'll implement InvocationHandler that logs before/after, measures duration, and rethrows the original exception. Then I'll build a proxy instance that implements the same interface(s) and delegates to the real object. This is zero-touch on the target and very handy for cross-cutting concerns.

Code:

```
import java.lang.reflect.*;
import java.util.Arrays;

// ====== Target interface + implementation ======
interface Worker {
    void executeTask(String jobId);
}

class ActualWorker implements Worker {
    public void executeTask(String jobId) {
        System.out.println("Executing task for " + jobId + " ...");
    }
}

// ====== Logging proxy handler ======
class LoggingInvocationHandler implements InvocationHandler {
    private final Object target;

    public LoggingInvocationHandler(Object target) {
        this.target = target;
    }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
        String sig = target.getClass().getSimpleName() + "#" + method.getName();
        String argStr = args == null ? "[]" : Arrays.toString(args);
        long start = System.nanoTime();

        System.out.printf("[ENTER] %s args=%s%n", sig, argStr);
        try {
            Object result = method.invoke(target, args);
```

```
            long tookMs = (System.nanoTime() - start) / 1_000_000;
            System.out.printf("[EXIT ] %s took=%dms%n", sig, tookMs);
            return result;
        } catch (InvocationTargetException ite) {
            long tookMs = (System.nanoTime() - start) / 1_000_000;
            Throwable cause = ite.getTargetException();
            System.out.printf("[ERROR] %s took=%dms ex=%s%n", sig, tookMs,
cause);
            throw cause; // preserve original semantics
        }
    }
}

// ====== Factory helper ======
class Proxies {
    @SuppressWarnings("unchecked")
    public static <T> T withLogging(T target, Class<T> iface) {
        return (T) Proxy.newProxyInstance(
                iface.getClassLoader(),
                new Class<?>[]{ iface },
                new LoggingInvocationHandler(target)
        );
    }
}

// ====== Demo ======
public class ProxyDemo {
    public static void main(String[] args) {
        Worker worker = new ActualWorker();
        Worker logged = Proxies.withLogging(worker, Worker.class);

        // Calls go through the proxy; ActualWorker isn't modified
        logged.executeTask("JOB-42");
    }
}
```

Reasoning to interviewer:

- JDK dynamic proxies work for interfaces; the proxy implements the same interface and delegates.
- Centralized InvocationHandler means I can add timing, argument logging, and error logging consistently.
- I unwrap InvocationTargetException so callers see the original failure type.
- This is the same foundation many AOP and RPC frameworks use internally.

Future scope:

- If targets are classes (no interface), use bytecode tools (CGLIB/ByteBuddy) or Spring AOP.
- Add annotation-driven behavior: log only methods annotated with @LogExecution.
- Include correlation IDs via MDC (SLF4J) and structured logging.
- Chain handlers (logging, metrics, tracing) for richer cross-cutting concerns.

Scenario 31: Designing a REST API router using annotations

You want a tiny, framework-free way to map HTTP requests like GET /users/42 to Java methods, using annotations to declare routes.

Answer:

I'll define @GET and @POST annotations that carry a value() path. At startup, a router scans handler classes, builds a registry keyed by (HTTP method, normalized path pattern), and supports simple path parameters (e.g., /users/{id}). On each request, the router matches the HTTP verb and path, extracts any path variables, and invokes the target method, passing variables by parameter name and type.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;
import java.util.regex.*;
import java.util.stream.Collectors;

// --- Annotations ---
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD)
@interface GET { String value(); }

@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD)
@interface POST { String value(); }

// --- HTTP-like request/response DTOs for demo ---
class Request {
    final String method; // "GET", "POST"
    final String path;   // e.g., "/users/42"
    final Map<String, String> query = new HashMap<>();
    Request(String method, String path) { this.method = method; this.path = path; }
}
class Response {
    int status = 200;
    String body = "";
}

// --- Example API with path params ---
class UserApi {
    @GET("/users")
    public String fetchUsers() {
        return "all users";
```

```
    }

    @GET("/users/{id}")
    public String getUser(String id) { // id bound by name
        return "user#" + id;
    }

    @POST("/users")
    public String addUser() {
        return "created user";
    }
}

// --- Router that supports path params like /users/{id} ---
class RequestRouter {

    // A route entry with compiled regex and parameter names
    static class Route {
        final String httpMethod;
        final Pattern pathRegex;
        final List<String> pathParamNames;
        final Object instance;
        final Method method;

        Route(String httpMethod, String template, Object instance, Method method) {
            this.httpMethod = httpMethod;
            this.instance = instance;
            this.method = method;

            // Convert "/users/{id}" -> "^/users/(?<id>[^/]+)$"
            List<String> names = new ArrayList<>();
            String regex = Arrays.stream(template.split("/"))
                    .filter(s -> !s.isEmpty())
                    .map(segment -> {
                        if (segment.startsWith("{") && segment.endsWith("}")) {
                            String name = segment.substring(1, segment.length() - 1);
                            names.add(name);
                            return "(?<" + name + ">[^/]+)";
                        }
                        return Pattern.quote(segment);
                    })
                    .collect(Collectors.joining("/", "^/", "$"));
            this.pathRegex = Pattern.compile(regex);
            this.pathParamNames = names;
        }

        boolean matches(Request req) {
            return httpMethod.equalsIgnoreCase(req.method) &&
pathRegex.matcher(req.path).matches();
        }
```

```
        Map<String, String> extractPathParams(String path) {
            Matcher m = pathRegex.matcher(path);
            Map<String, String> map = new HashMap<>();
            if (m.matches()) {
                for (String n : pathParamNames) map.put(n, m.group(n));
            }
            return map;
        }
    }

    private final List<Route> routes = new ArrayList<>();

    // Register all annotated methods on a handler instance
    public void register(Object handler) {
        for (Method m : handler.getClass().getDeclaredMethods()) {
            if (m.isAnnotationPresent(GET.class)) {
                routes.add(new Route("GET", m.getAnnotation(GET.class).value(),
handler, m));
            } else if (m.isAnnotationPresent(POST.class)) {
                routes.add(new Route("POST", m.getAnnotation(POST.class).value(),
handler, m));
            }
        }
    }

    // Dispatch an incoming request
    public Response dispatch(Request req) {
        Response res = new Response();

        // 1) Find all routes sharing the URI ignoring method to differentiate 404 vs
405
        List<Route> samePath = routes.stream()
                .filter(r -> r.pathRegex.matcher(req.path).matches())
                .collect(Collectors.toList());

        // 2) Find exact match on method+path
        for (Route r : routes) {
            if (r.matches(req)) {
                try {
                    Map<String, String> pathVars = r.extractPathParams(req.path);
                    Object result = invokeWithParams(r.instance, r.method, pathVars);
                    res.status = 200;
                    res.body = (result == null) ? "" : result.toString();
                    return res;
                } catch (InvocationTargetException ite) {
                    res.status = 500;
                    res.body = "Server error: " + ite.getTargetException().getMessage();
                    return res;
                } catch (Exception e) {
```

```
                res.status = 500;
                res.body = "Server error: " + e.getMessage();
                return res;
            }
        }
    }

    // If path matched but method didn't, return 405; else 404
    if (!samePath.isEmpty()) {
        res.status = 405;
        res.body = "Method Not Allowed";
    } else {
        res.status = 404;
        res.body = "Not Found";
    }
    return res;
  }

  // Bind method parameters by name from path variables (String-only for demo)
  private Object invokeWithParams(Object target, Method method, Map<String,
String> pathVars) throws Exception {
    Parameter[] params = method.getParameters();
    Object[] args = new Object[params.length];
    for (int i = 0; i < params.length; i++) {
        String name = params[i].getName(); // requires -parameters at compile time
for real names
        // Fallback: single param gets the first path var
        if (params.length == 1 && !pathVars.isEmpty() && (name == null ||
name.isEmpty())) {
            args[i] = pathVars.values().iterator().next();
        } else {
            args[i] = pathVars.get(name);
        }
    }
    return method.invoke(target, args);
  }
}

// --- Demo ---
public class RestApiDemo {
  public static void main(String[] args) {
    RequestRouter router = new RequestRouter();
    router.register(new UserApi());

    Response r1 = router.dispatch(new Request("GET", "/users"));
    System.out.println("GET /users -> " + r1.status + " " + r1.body);

    Response r2 = router.dispatch(new Request("GET", "/users/42"));
    System.out.println("GET /users/42 -> " + r2.status + " " + r2.body);
```

```
        Response r3 = router.dispatch(new Request("POST", "/users"));
        System.out.println("POST /users -> " + r3.status + " " + r3.body);

        Response r4 = router.dispatch(new Request("DELETE", "/users"));
        System.out.println("DELETE /users -> " + r4.status + " " + r4.body);
    }
}
```

Reasoning to interviewer:

- Annotations declare the routing contract right on the handler methods; the router centralizes discovery and dispatch.
- Compiling templates with named groups lets me extract path variables without a full router library.
- Differentiating 404 (no route for path) vs 405 (path exists but method not allowed) mirrors real REST semantics.
- Parameter binding keeps method signatures clean; for production you'd add converters (String→int/UUID) and query/header/body binding.

Future scope:

- Support more verbs (PUT, DELETE, PATCH) and content negotiation.
- Add parameter annotations like @PathVar("id"), @Query("q"), @Body for robust binding without relying on -parameters.
- Middleware filters for auth, CORS, and rate limiting.
- Error mappers to serialize exceptions into consistent JSON problem responses.

Scenario 32: Building a transaction management system using custom annotations

You want a simple way to wrap certain methods in a “begin–commit–rollback” boundary, without sprinkling transaction code into every service.

Answer:

I’ll introduce @Transactional and a tiny TransactionManager with begin()/commit()/rollback(); the controller finds annotated methods, begins a transaction, invokes the method, commits on success, and rolls back on exceptions. I’ll keep state in a ThreadLocal to simulate per-request transactions and allow a rollbackFor attribute to decide which exceptions trigger rollbacks.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;

// --- Annotation ---
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Transactional {
    Class<? extends Throwable>[] rollbackFor() default { RuntimeException.class,
Error.class };
}

// --- Dumb in-memory transaction manager (demo) ---
class TransactionManager {
    private static final ThreadLocal<Boolean> ACTIVE = ThreadLocal.withInitial(() -
> false);

    public void begin() {
        if (ACTIVE.get()) throw new IllegalStateException("Nested tx not supported in
demo");
        ACTIVE.set(true);
        System.out.println("[TX] BEGIN");
    }

    public void commit() {
        if (!ACTIVE.get()) throw new IllegalStateException("No active tx");
        System.out.println("[TX] COMMIT");
        ACTIVE.remove();
    }
```

```
    public void rollback() {
        if (!ACTIVE.get()) throw new IllegalStateException("No active tx");
        System.out.println("[TX] ROLLBACK");
        ACTIVE.remove();
    }

    public boolean isActive() { return ACTIVE.get(); }
}

// --- Service using @Transactional ---
class TransactionalService {

    @Transactional // default: rollback on RuntimeException/Error
    public void processTransaction() {
        System.out.println("Executing transactional operation...");
        // imagine DB writes here
    }

    @Transactional(rollbackFor = { Exception.class }) // rollback even on checked
exceptions
    public void processAndFailChecked() throws Exception {
        System.out.println("Executing operation (will throw checked)");
        throw new Exception("Checked failure");
    }
}

// --- Controller that wraps calls in tx boundaries ---
class TransactionController {
    private static final TransactionManager TM = new TransactionManager();

    public static void executeWithTransaction(Object target) {
        for (Method m : target.getClass().getDeclaredMethods()) {
            if (!m.isAnnotationPresent(Transactional.class)) continue;

            Transactional meta = m.getAnnotation(Transactional.class);
            try {
                TM.begin();
                m.invoke(target); // no args demo
                TM.commit();
            } catch (InvocationTargetException ite) {
                Throwable cause = ite.getTargetException();
                if (shouldRollback(meta, cause)) {
                    TM.rollback();
                } else {
                    TM.commit(); // not listed -> commit (demo behavior)
                }
                // surface the exception to caller or log
                System.out.println("[TX] Exception: " +
cause.getClass().getSimpleName() + " -> " + cause.getMessage());
```

```
            } catch (Exception e) {
                // reflection errors
                if (TM.isActive()) TM.rollback();
                System.out.println("[TX] Reflection error: " + e);
            }
        }
    }

    private static boolean shouldRollback(Transactional meta, Throwable t) {
        for (Class<? extends Throwable> c : meta.rollbackFor()) {
            if (c.isInstance(t)) return true;
        }
        return false;
    }
}

// --- Demo ---
public class TransactionDemo {
    public static void main(String[] args) {
        TransactionalService service = new TransactionalService();
        TransactionController.executeWithTransaction(service);
    }
}
```

Reasoning to interviewer:

- The annotation marks intent; the controller centralizes the policy—classic separation of concerns.
- rollbackFor mirrors real frameworks (Spring) so teams can declare exactly which failures should revert changes.
- ThreadLocal simulates per-request transaction scope; in practice this wraps a real DB transaction or unit-of-work.
- Commit/rollback logging makes flow visible for demos and testing.

Future scope:

- Add propagation (REQUIRED, REQUIRES_NEW) and nested transactions via savepoints.
- Integrate with a real datastore (JDBC) and ensure rollback only happens if work occurred.
- Hook into AOP/dynamic proxies so any call through the interface is auto-wrapped.

- Include readOnly flag to hint optimizations and timeout to prevent long-running transactions.

Scenario 33: Role-based access control using a custom annotation

You want a clean, declarative way to restrict who can call certain methods—admin-only operations, manager-only reports, etc.—without scattering role checks everywhere.

Answer:

I'll define an @Authorize annotation that lists allowed roles and support it at both class and method levels (method overrides class). At runtime, an authorization layer resolves the "current user" from a SecurityContext and checks hasAnyRole(allowed). If authorization fails, it throws a clear exception before the method runs. I'll use an enum for roles to avoid string typos and key decisions like "method annotation overrides class annotation" to keep intent obvious.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.EnumSet;

// 1) Roles as an enum to avoid string typos
enum Role { USER, MANAGER, ADMIN }

// 2) Thread-local security context (simple demo)
class SecurityContext {
    private static final ThreadLocal<Role> CURRENT = new ThreadLocal<>();
    public static void set(Role r) { CURRENT.set(r); }
    public static Role currentRole() { return CURRENT.get(); }
}

// 3) Annotation usable on type and method; method-level overrides type-level
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
@interface Authorize {
    Role[] allowedRoles();
}

// 4) Example service
@Authorize(allowedRoles = {Role.MANAGER, Role.ADMIN}) // default for all
methods
class ReportService {

    public void viewDashboard() {
        System.out.println("Dashboard visible to MANAGER/ADMIN");
    }
```

```
    @Authorize(allowedRoles = {Role.ADMIN}) // stricter for this method
    public void deleteAllReports() {
        System.out.println("All reports deleted (ADMIN only)");
    }
}

// 5) Authorization manager: method-level overrides class-level
class AccessControl {

    public static Object authorizeAndInvoke(Object target, Method method, Object...
args) throws Throwable {
        // compute effective allowed roles
        Authorize methodAnno = method.getAnnotation(Authorize.class);
        Authorize classAnno  = target.getClass().getAnnotation(Authorize.class);

        EnumSet<Role> allowed = EnumSet.noneOf(Role.class);
        if (classAnno != null) for (Role r : classAnno.allowedRoles()) allowed.add(r);
        if (methodAnno != null) { // override if present
            allowed.clear();
            for (Role r : methodAnno.allowedRoles()) allowed.add(r);
        }

        Role current = SecurityContext.currentRole();
        if (!allowed.isEmpty() && (current == null || !allowed.contains(current))) {
            throw new IllegalAccessException("Access denied for role: " + current + "
to " + method.getName());
        }

        try {
            return method.invoke(target, args);
        } catch (InvocationTargetException ite) {
            throw ite.getTargetException();
        }
    }
}

// 6) Demo
public class AccessControlDemo {
    public static void main(String[] args) throws Throwable {
        ReportService service = new ReportService();

        Method dashboard = ReportService.class.getMethod("viewDashboard");
        Method purge     = ReportService.class.getMethod("deleteAllReports");

        SecurityContext.set(Role.MANAGER);
        AccessControl.authorizeAndInvoke(service, dashboard); // OK
        try {
            AccessControl.authorizeAndInvoke(service, purge);   // denied
        } catch (IllegalAccessException e) {
            System.out.println(e.getMessage());
```

```
        }

        SecurityContext.set(Role.ADMIN);
        AccessControl.authorizeAndInvoke(service, purge);      // OK
    }
}
```

Reasoning to interviewer:

- The annotation lives next to the code it protects; the check lives in one place.
- Method-level metadata overriding class-level defaults mirrors common frameworks.
- Using an enum for roles prevents string drift and enables fast EnumSet checks.
- The guard fails fast, before any side effects happen.

Future scope:

- Add logical policies like allOf() vs anyOf() roles.
- Support permissions/scopes (fine-grained) alongside roles (coarse-grained).
- Plug into real auth (JWT/OIDC) to populate SecurityContext.
- Emit audit logs on allow/deny and integrate with rate limiting for sensitive ops.

Scenario 34: Method result caching using custom annotations

You want to memoize expensive calls so repeated invocations with the same arguments return fast.

Answer:
I'll add a @Cacheable annotation and a cache layer that:

1. builds a stable key from the full method signature + arguments,
2. uses a thread-safe map to store results, and
3. returns cached values on hits.
   I'll use ConcurrentHashMap with computeIfAbsent so only one thread does the work per key. I'll also show TTL and a simple key composer to avoid collisions.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.Arrays;
import java.util.Objects;
import java.util.concurrent.*;

// 1) Annotation with optional TTL
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Cacheable {
    long ttlMs() default 0; // 0 = no expiry
}

// 2) Example service
class CalculationService {
    @Cacheable(ttlMs = 2_000) // cache for 2 seconds
    public int heavyCalculation(int param) {
        System.out.println("Performing heavy calculation...");
        try { Thread.sleep(300); } catch (InterruptedException ignored) {}
        return param * 42;
    }
}

// 3) Cache entry with expiry
class Entry {
    final Object value;
    final long   expiresAt; // epoch ms; 0 means no expiry
    Entry(Object value, long expiresAt) {
        this.value = value; this.expiresAt = expiresAt;
    }
```

```
    boolean expired() { return expiresAt > 0 && System.currentTimeMillis() >
expiresAt; }
}

// 4) Cache proxy
class CacheProxy {
    private static final ConcurrentHashMap<String, Entry> CACHE = new
ConcurrentHashMap<>();

    public static Object invokeWithCache(Object target, Method method, Object...
args) throws Throwable {
        Cacheable meta = method.getAnnotation(Cacheable.class);
        if (meta == null) {
            try { return method.invoke(target, args); }
            catch (InvocationTargetException ite) { throw ite.getTargetException(); }
        }

        final String key = key(method, args);
        final long ttl = meta.ttlMs();

        // computeIfAbsent doesn't help with expiry; handle it explicitly
        Entry e = CACHE.get(key);
        if (e != null && !e.expired()) return e.value;

        // Only one builder per key; others spin briefly if racing
        synchronized (CACHE.computeIfAbsent(key + ":lock", __ -> new Entry(new
Object(), 0)).value) {
            // recheck after lock to avoid duplicate work
            e = CACHE.get(key);
            if (e != null && !e.expired()) return e.value;

            try {
                Object result = method.invoke(target, args);
                long exp = ttl > 0 ? System.currentTimeMillis() + ttl : 0;
                CACHE.put(key, new Entry(result, exp));
                return result;
            } catch (InvocationTargetException ite) {
                throw ite.getTargetException();
            }
        }
    }

    // Key: declaring class + method + param types + arg values
    private static String key(Method m, Object[] args) {
        return m.getDeclaringClass().getName() + "#" + m.getName() + "(" +

Arrays.stream(m.getParameterTypes()).map(Class::getName).reduce((a,b)-
>a+","+b).orElse("") +
                "):" + Arrays.deepHashCode(args);
    }
```

```
}

// 5) Demo
public class CacheDemo {
    public static void main(String[] args) throws Throwable {
        CalculationService svc = new CalculationService();
        Method heavy = CalculationService.class.getMethod("heavyCalculation",
int.class);

        System.out.println(CacheProxy.invokeWithCache(svc, heavy, 7)); // computes
& caches
        System.out.println(CacheProxy.invokeWithCache(svc, heavy, 7)); // cached
        Thread.sleep(2200);
        System.out.println(CacheProxy.invokeWithCache(svc, heavy, 7)); // expired -
> recompute
    }
}
```

Reasoning to interviewer:

- The cache key includes method signature + arguments to avoid collisions across overloads.
- ConcurrentHashMap makes it thread-safe; the per-key lock prevents “cache stampede” under contention.
- TTL keeps the cache fresh; expired entries recompute lazily.
- Exceptions are unwrapped so callers see the original failure type, not the reflection wrapper.

Future scope:

- Add @CacheEvict/@CachePut for invalidation and refresh.
- Pluggable caches (Caffeine/Guava/Redis) with size limits and eviction policies.
- Fine-grained key control with @CacheKey on parameters or a custom key generator.
- Cache nulls and failures separately (with a short TTL) to avoid repeated hammering on broken calls.

Scenario 35: Defining how long annotations are retained

When designing custom annotations, you need to control how long they remain in the lifecycle (source, class, or runtime). This choice affects both performance and use cases.

Answer:

The retention duration is controlled by the meta-annotation @Retention. It accepts a RetentionPolicy value:

- SOURCE: annotation is discarded by the compiler; useful for documentation or compile-time tools like @Override.
- CLASS: annotation is kept in the .class file but not loaded into the JVM; useful when tools like bytecode analyzers or build tools need to see it, but runtime logic doesn't.
- RUNTIME: annotation is available at runtime via reflection; required for frameworks like Spring or JUnit that inspect annotations dynamically.

For a logging marker, you want RUNTIME so that reflection can pick up annotated methods during execution.

Code:

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)   // available during runtime
@Target(ElementType.METHOD)           // can only be used on methods
@interface LogMarker {}

class LoggerExample {
    @LogMarker
    public void performTask() {
        System.out.println("Executing task with logging enabled");
    }
}
```

Reasoning to interviewer:

- @Retention(RUNTIME) ensures logging frameworks or reflection utilities can see the marker while running.
- Without this, annotations vanish after compilation and reflection would never find them.
- Restricting @Target to METHOD avoids accidental misuse on fields or classes.

Future scope:

- Combine with @Inherited if you want child classes to automatically inherit class-level markers.
- Use retention SOURCE for compile-time validation annotations (like custom lint checks).
- Use retention CLASS for bytecode weaving or build-time tools.

Scenario 36: Using reflection to retrieve all methods of a class

Sometimes you need an inspection tool to see every method in a class—for documentation generators, debuggers, or dynamic frameworks.

Answer:

The Reflection API exposes methods like getDeclaredMethods() (all declared, including private ones) and getMethods() (all public methods, including inherited). You can iterate over them to build documentation, validate naming conventions, or even generate proxies.

Code:

```
import java.lang.reflect.Method;
import java.lang.reflect.Modifier;

class SampleClass {
    public void firstMethod() {}
    private void secondMethod() {}
    protected String thirdMethod(int x) { return "Value: " + x; }
}

public class ReflectionDemo {
    public static void main(String[] args) {
        Class<?> clazz = SampleClass.class;

        System.out.println("Declared methods in " + clazz.getSimpleName() + ":");
        for (Method m : clazz.getDeclaredMethods()) {
            System.out.println(
                Modifier.toString(m.getModifiers()) + " " +
                m.getReturnType().getSimpleName() + " " +
                m.getName()
            );
        }
    }
}
```

Reasoning to interviewer:

- getDeclaredMethods() gives all methods defined in that class (private, protected, package, and public).
- getMethods() differs by including inherited methods from superclasses/interfaces.

- Using Modifier.toString() and getReturnType() makes the output more descriptive, not just method names.
- This is foundational for building ORMs, serializers, or annotation processors that depend on method metadata.

Future scope:

- Extend utility to also list parameters (getParameters()), exceptions (getExceptionTypes()), and annotations.
- Useful for auto-documenting APIs, generating UML diagrams, or enforcing coding guidelines.
- Combine with custom annotations to filter methods for specific purposes (e.g., only list @TestMethod methods).

Scenario 37: Enforcing non-null fields with a custom annotation

You want a simple, uniform way to guarantee certain fields are never null and fail fast if they are.

Answer:

I'll introduce a @NonNull field annotation and a small validator that walks an object's fields via reflection. For each @NonNull field, it checks the value; if null, it throws an IllegalArgumentException that clearly names the field and owning class. I'll support private fields (using setAccessible(true)), nested objects (optional deep mode), and collections/maps where the reference itself must not be null.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.Collection;
import java.util.Map;

// 1) Annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NonNull {
    boolean deep() default false; // optionally validate nested objects too
}

// 2) Example model
class Address {
    @NonNull String city;
    String pin;
}

class Person {
    @NonNull String fullName;
    @NonNull(deep = true) Address primaryAddress; // deep-check nested fields
marked @NonNull
}

// 3) Validator
class NullValidator {

    public static void validate(Object obj) {
        if (obj == null) throw new IllegalArgumentException("Object to validate is
null");
        validateObject(obj, obj.getClass().getSimpleName());
    }
```

```
    private static void validateObject(Object obj, String path) {
        Class<?> type = obj.getClass();
        for (Field f : type.getDeclaredFields()) {
            f.setAccessible(true);
            NonNull nn = f.getAnnotation(NonNull.class);
            if (nn == null) continue;

            Object val;
            try {
                val = f.get(obj);
            } catch (IllegalAccessException e) {
                throw new IllegalStateException("Cannot read field: " + path + "." +
f.getName(), e);
            }

            if (val == null) {
                throw new IllegalArgumentException("@" +
NonNull.class.getSimpleName() +
                        " violation at " + path + "." + f.getName() + " (must not be null)");
            }

            // Optional deep validation: only recurse into user types (not JDK simple
types)
            if (nn.deep() && isUserType(val.getClass())) {
                validateObject(val, path + "." + f.getName());
            }

            // For collections/maps, we only assert reference non-null here (content
checks are policy-specific)
            if (val instanceof Collection<?> || val instanceof Map<?, ?>) {
                // could extend to check elements/keys/values with a different annotation
            }
        }
    }

    private static boolean isUserType(Class<?> cls) {
        return !(cls.isPrimitive() ||
                 cls.getName().startsWith("java.") ||
                 cls.isEnum());
    }
}

// 4) Demo
public class ValidationDemo {
    public static void main(String[] args) {
        Person p = new Person();
        p.fullName = "Asha";
        p.primaryAddress = new Address();
        // p.primaryAddress.city remains null -> deep violation
```

```
        NullValidator.validate(p); // throws: @NonNull violation at
Person.primaryAddress.city
    }
}
```

Reasoning to interviewer:

- The annotation declares intent where it matters (on the field); the validator enforces it uniformly.
- I fail fast with a readable path (Person.primaryAddress.city) so troubleshooting is quick.
- deep=true lets teams opt-in to nested validation without forcing it everywhere.
- Using reflection on private fields prevents leaking validation concerns into domain code.

Future scope:

- Add @NonEmpty/@NotBlank for String, and @Size(min, max) for collections.
- Provide a Violations list instead of throwing on first error, so UIs can display all issues at once.
- Cache reflective metadata per class to avoid repeated reflection overhead on hot paths.
- Integrate with Bean Validation (JSR-380) if you later want a standards-based approach.

Scenario 38: Adding metadata to fields for documentation

You want human-readable descriptions and defaults attached to fields so tools can generate docs or UIs automatically.

Answer:

I'll define @FieldDescription(info, defaultValue) with runtime retention on fields. A small extractor will scan a class and build a metadata table (or JSON) listing field name, type, description, and suggested default. This is handy for config pages, API docs, or admin consoles.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

// 1) Annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface FieldDescription {
    String info();
    String defaultValue() default "undefined";
}

// 2) Example model
class ProductItem {
    @FieldDescription(info = "Name of the product")
    private String productName;

    @FieldDescription(info = "Price of the product", defaultValue = "0.0")
    private double productPrice;

    @FieldDescription(info = "ISO currency code", defaultValue = "USD")
    private String currency;
}

// 3) Doc entry + extractor
class FieldDoc {
    final String field;
    final String type;
    final String info;
    final String defaultValue;

    FieldDoc(String field, String type, String info, String defaultValue) {
```

```
        this.field = field; this.type = type; this.info = info; this.defaultValue =
defaultValue;
    }

    @Override public String toString() {
        return field + " : " + type + " — " + info + " (default=" + defaultValue + ")";
    }
}

class Documentation {
    public static List<FieldDoc> describe(Class<?> clazz) {
        List<FieldDoc> docs = new ArrayList<>();
        for (Field f : clazz.getDeclaredFields()) {
            FieldDescription fd = f.getAnnotation(FieldDescription.class);
            if (fd == null) continue;
            docs.add(new FieldDoc(
                    f.getName(),
                    f.getType().getSimpleName(),
                    fd.info(),
                    fd.defaultValue()
            ));
        }
        return docs;
    }
}

// 4) Demo
public class FieldDocDemo {
    public static void main(String[] args) {
        List<FieldDoc> docs = Documentation.describe(ProductItem.class);
        docs.forEach(System.out::println);
        // Example output:
        // productName : String — Name of the product (default=undefined)
        // productPrice : double — Price of the product (default=0.0)
        // currency : String — ISO currency code (default=USD)
    }
}
```

Reasoning to interviewer:

- The annotation keeps documentation close to the field it describes, reducing drift between code and docs.
- A tiny reflection utility produces structured metadata that downstream tools can render as HTML/Markdown/JSON.
- Keeping runtime retention enables both build-time generators and live admin UIs to read metadata.

Future scope:

- Add attributes like example(), since(), deprecatedReason(), or sensitive() for redaction.
- Generate OpenAPI-like schemas for REST, or form definitions for UI scaffolding.
- Localize info() via keys that a doc generator resolves from resource bundles.
- Combine with validation annotations to emit richer, constraint-aware documentation.

Scenario 39: Defining a validation annotation with parameters (max string length)

You want a declarative way to cap string length on fields and fail fast with a clear message.

Answer:

I'll create a parameterized @MaxLen annotation and a validator that reflects over fields, checks the annotation's value() (the max length), and records violations. I prefer returning a list of errors (or throwing one exception containing them) so the caller gets a full report in one pass. I'll also handle nulls safely and private fields via setAccessible(true).

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

// 1) Annotation with a customizable error message
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLen {
    int value();
    String message() default "exceeds max length";
}

// 2) Example domain
class Account {
    @MaxLen(value = 12, message = "username too long")
    private String username;

    public Account(String username) { this.username = username; }
}

// 3) Validation error carrier
class Violation {
    final String field;
    final String detail;
    Violation(String field, String detail) { this.field = field; this.detail = detail; }
    @Override public String toString() { return field + ": " + detail; }
}

// 4) Validator that aggregates violations
class LengthValidator {
```

```
    public static void validate(Object bean) {
        List<Violation> out = new ArrayList<>();
        for (Field f : bean.getClass().getDeclaredFields()) {
            MaxLen ann = f.getAnnotation(MaxLen.class);
            if (ann == null) continue;

            f.setAccessible(true);
            try {
                Object val = f.get(bean);
                if (val == null) continue;                // null is fine here; combine with
@NonNull if needed
                if (!(val instanceof CharSequence)) {
                    out.add(new Violation(f.getName(), "@MaxLen supports
CharSequence types only"));
                    continue;
                }
                int max = ann.value();
                int len = ((CharSequence) val).length();
                if (len > max) {
                    out.add(new Violation(
                        f.getName(),
                        ann.message() + " (len=" + len + ", max=" + max + ")"
                    ));
                }
            } catch (IllegalAccessException e) {
                out.add(new Violation(f.getName(), "unreadable field"));
            }
        }
        if (!out.isEmpty()) {
            StringBuilder sb = new StringBuilder("Validation failed:\n");
            out.forEach(v -> sb.append(" - ").append(v).append('\n'));
            throw new IllegalArgumentException(sb.toString());
        }
    }
}

// 5) Demo
public class MaxLenDemo {
    public static void main(String[] args) {
        Account ok = new Account("shortName");
        Account bad = new Account("this_is_too_long");

        LengthValidator.validate(ok); // passes

        try {
            LengthValidator.validate(bad); // throws aggregated error
        } catch (IllegalArgumentException ex) {
            System.out.println(ex.getMessage());
        }
    }
```

```
}
```

Reasoning to interviewer:

- The annotation expresses the rule where the data lives; the validator centralizes enforcement.
- Aggregating violations is friendlier for UIs than failing on the first field.
- I gate on CharSequence so it also supports StringBuilder, etc., and I tolerate nulls unless combined with a separate @NonNull.
- Using setAccessible(true) lets us validate private fields without modifying domain classes.

Future scope:

- Add @MinLen, @Pattern, and compose rules (repeatable annotations).
- Cache reflection metadata (fields + annotations) per class to reduce overhead on hot paths.
- Plug into a broader framework (JSR-380/Bean Validation) by adapting annotations to standard constraints.
- Support nested objects (walk graphs) and collection element constraints with a companion @Each annotation.

Scenario 40: Creating instances using reflection and default constructor

You want to instantiate types dynamically without compile-time knowledge of their constructors.

Answer:

If a class has a public no-arg constructor, use getConstructor().newInstance(). If it's non-public, fall back to getDeclaredConstructor() and make it accessible. I'll also show safe exception unwrapping and a clear error if no default constructor exists (common in immutable designs), plus a convenience that tries a static factory if provided.

Code:

```
import java.lang.reflect.Constructor;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;

// 1) Basic factory: default ctor preferred, then non-public default ctor
class InstanceCreator {

    public static <T> T create(Class<T> type) {
        try {
            // Try public no-arg
            Constructor<T> c = type.getConstructor();
            return c.newInstance();
        } catch (NoSuchMethodException e) {
            // Try non-public no-arg
            try {
                Constructor<T> c = type.getDeclaredConstructor();
                c.setAccessible(true);
                return c.newInstance();
            } catch (NoSuchMethodException e2) {
                // Fall through to factory search
                return tryStaticFactory(type);
            } catch (InstantiationException | IllegalAccessException |
InvocationTargetException ex) {
                throw wrap("Failed invoking non-public default constructor", ex);
            }
        } catch (InstantiationException | IllegalAccessException |
InvocationTargetException ex) {
            throw wrap("Failed invoking public default constructor", ex);
        }
    }
```

```
    // 2) Optional: look for a conventional static factory like "create"/"newInstance"
returning the type
    private static <T> T tryStaticFactory(Class<T> type) {
        for (String name : new String[]{"create", "of", "newInstance"}) {
            try {
                Method m = type.getDeclaredMethod(name);
                if (!type.isAssignableFrom(m.getReturnType())) continue;
                m.setAccessible(true);
                @SuppressWarnings("unchecked")
                T instance = (T) m.invoke(null);
                return instance;
            } catch (NoSuchMethodException ignore) {
            } catch (IllegalAccessException | InvocationTargetException ex) {
                throw wrap("Failed invoking static factory " + name + "()", ex);
            }
        }
        throw new IllegalStateException("No default constructor or supported static
factory for " + type.getName());
    }

    private static RuntimeException wrap(String msg, Exception ex) {
        Throwable cause = (ex instanceof InvocationTargetException)
                ? ((InvocationTargetException) ex).getTargetException()
                : ex;
        return new RuntimeException(msg + ": " + cause, cause);
    }
}

// 3) Demos
class Sample {
    public Sample() { System.out.println("Sample default constructor called"); }
}

class HiddenCtor {
    private HiddenCtor() { System.out.println("Hidden default ctor called"); }
}

class FactoryOnly {
    private FactoryOnly() {}
    static FactoryOnly create() { System.out.println("FactoryOnly.create() called");
return new FactoryOnly(); }
}

public class ReflectionCtorDemo {
    public static void main(String[] args) {
        Sample s = InstanceCreator.create(Sample.class);           // public no-arg
        HiddenCtor h = InstanceCreator.create(HiddenCtor.class);   // non-public no-
arg
        FactoryOnly f = InstanceCreator.create(FactoryOnly.class); // static factory
    }
```

```
}
```

Reasoning to interviewer:

- getConstructor() finds only public no-arg; getDeclaredConstructor() lets me access private ones (with setAccessible(true)).
- Unwrapping InvocationTargetException preserves the original cause for debugging.
- A static factory fallback mirrors common patterns in immutable types where no default constructor exists.
- The API returns a strongly typed instance and surfaces clear errors when instantiation isn’t possible.

Future scope:

- Support parameterized construction by accepting a map of argument values or a dependency provider (ties into DI).
- Cache constructors/method handles (MethodHandles.Lookup) to reduce reflective overhead.
- Respect security managers or module boundaries (Java 9+) where setAccessible(true) may be restricted.
- Add bean initialization hooks (e.g., @PostConstruct) and lifecycle management if used inside a container.

Scenario 41: Enforcing deprecated API usage warnings

You want to clearly mark old code paths, nudge callers away from them, but still allow controlled use in a few places without drowning in warnings.

Answer:

I'll use Java's built-in @Deprecated (with since and forRemoval) on the API I'm phasing out. Call sites that must temporarily keep using it can wrap the usage with @SuppressWarnings("deprecation"). If I've marked an API with forRemoval=true on newer JDKs, the compiler also raises a "removal" warning—so for very targeted silencing I'll use @SuppressWarnings({"deprecation","removal"}) at the narrowest scope (method or local block). I'll also add a Javadoc @deprecated tag with the migration path so IDEs show the guidance inline.

Code:

```
// Library / legacy module
class LegacyAPI {

    /**
     * @deprecated Use newMethod() instead. This will be removed after 2025-12.
     */
    @Deprecated(since = "1.4", forRemoval = true)
    public void oldMethod() {
        System.out.println("Deprecated path");
    }

    public void newMethod() {
        System.out.println("Modern path");
    }
}

// Calling code that must still use it in one spot
class ReportJob {

    // Narrow suppression to just this method, not the whole class
    @SuppressWarnings({"deprecation", "removal"})
    void runOnceForBackwardCompat() {
        LegacyAPI api = new LegacyAPI();
        api.oldMethod();  // warning suppressed here only
    }

    void normalRun() {
        new LegacyAPI().newMethod(); // preferred
    }
}
```

Reasoning to interviewer:

- @Deprecated(since, forRemoval) communicates intent and timeline—this is more than a generic deprecation.
- I silence warnings as locally as possible, so the rest of the project still gets useful compiler signals.
- Javadoc @deprecated plus a replacement method makes migration self-service for callers.

Future scope:

- Add build rules (e.g., Error Prone/Checkstyle) to ban new usages of deprecated APIs outside approved modules.
- Emit runtime logs the first few uses in non-test environments to spot stragglers.
- Use @Deprecated(forRemoval=true) as the final step before actually deleting the code in the next major version.

Scenario 42: Creating a custom annotation to track API versioning

You want a simple, declarative way to tag classes/methods with the API version they belong to and optionally pick the right method at runtime for a requested version.

Answer:

I'll define @APIVersion with major/minor (or a String if you prefer). At runtime I can either:

1. scan classes to produce a report (docs, audits), or
2. route calls to the best-matching implementation for a requested version (e.g., pick the highest version ≤ requested).
   I'll keep retention RUNTIME so reflection can read it, and allow usage on both types and methods.

Code (version tagging + simple version-aware dispatcher):

```
import java.lang.annotation.*;
import java.lang.reflect.Method;
import java.util.Comparator;
import java.util.Optional;

@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
@interface APIVersion {
    int major();
    int minor() default 0;
}

// Example API with evolving methods
@APIVersion(major = 1, minor = 2)
class UserApi {

    @APIVersion(major = 1, minor = 0)
    public String getUserV1(String id) { return "v1 user:" + id; }

    @APIVersion(major = 1, minor = 3)
    public String getUserV1_3(String id) { return "v1.3 user:" + id; } // newer variant
}

// A tiny dispatcher that selects the best method for a requested version
class VersionedInvoker {

    static String invokeBest(Object target, String baseName, int reqMajor, int
reqMinor, Object... args) throws Exception {
        Method[] methods = target.getClass().getDeclaredMethods();
```

```
        Optional<Method> best = java.util.Arrays.stream(methods)
            .filter(m -> m.getName().startsWith(baseName) &&
m.isAnnotationPresent(APIVersion.class))
            .filter(m -> {
                APIVersion av = m.getAnnotation(APIVersion.class);
                // only methods not newer than requested
                return (av.major() < reqMajor) ||
                       (av.major() == reqMajor && av.minor() <= reqMinor);
            })
            .max(Comparator.comparingInt((Method m) ->
m.getAnnotation(APIVersion.class).major())
                 .thenComparingInt(m -> m.getAnnotation(APIVersion.class).minor()));

        Method method = best.orElseThrow(() ->
            new NoSuchMethodException("No suitable version for " + baseName + " at
" + reqMajor + "." + reqMinor));

        return (String) method.invoke(target, args);
    }
}

// Demo: report + dispatch
public class VersionScanner {
    public static void main(String[] args) throws Exception {
        // Reporting
        Class<UserApi> clazz = UserApi.class;
        if (clazz.isAnnotationPresent(APIVersion.class)) {
            APIVersion clsV = clazz.getAnnotation(APIVersion.class);
            System.out.println("Class API version: " + clsV.major() + "." + clsV.minor());
        }
        for (var m : clazz.getDeclaredMethods()) {
            if (m.isAnnotationPresent(APIVersion.class)) {
                APIVersion mv = m.getAnnotation(APIVersion.class);
                System.out.println(m.getName() + " -> " + mv.major() + "." + mv.minor());
            }
        }

        // Version-aware call routing
        UserApi api = new UserApi();
        System.out.println(VersionedInvoker.invokeBest(api, "getUser", 1, 2, "42")); //
picks v1.0
        System.out.println(VersionedInvoker.invokeBest(api, "getUser", 1, 3, "42")); //
picks v1.3
    }
}
```

Reasoning to interviewer:

- @APIVersion placed on class and method gives both a default (class level) and precise overrides (method level).
- The dispatcher demonstrates a practical use: pick the best implementation compatible with the client's requested version.
- Using integers avoids parsing complexities; the comparator logic is straightforward and easy to test.

Future scope:

- Make @APIVersion repeatable if you need to declare compatibility ranges.
- Generate API docs or change logs by scanning packages and emitting Markdown/HTML.
- Integrate with routing (e.g., X-API-Version header) so the server automatically selects the right method.
- Add @Deprecated alongside older @APIVersion methods to guide clients during migrations.

Scenario 43: Runtime processing of custom annotations for authorization

You want a declarative, centralized way to gate sensitive methods by user role, without sprinkling if/else checks across the codebase.

Answer:

I'll define a @RoleAllowed annotation that lists permitted roles. At runtime, a thin authorization layer reads the current user's role from a thread-local security context and checks it against the annotation. I'll support both class-level and method-level annotations (method overrides class) and use an enum for roles to avoid string typos. If access is denied, I'll fail fast with a clear exception before the method executes.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.EnumSet;

// Roles as an enum (prevents string drift)
enum Role { GUEST, USER, MANAGER, ADMIN }

// Thread-local security context (simulates per-request auth)
class SecurityContext {
    private static final ThreadLocal<Role> CURRENT = new ThreadLocal<>();
    public static void set(Role r) { CURRENT.set(r); }
    public static Role get() { return CURRENT.get(); }
}

// Annotation usable on type and method; method-level overrides class-level
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
@interface RoleAllowed {
    Role[] value();
}

// Example service
@RoleAllowed({Role.MANAGER, Role.ADMIN}) // default for all methods
class SecureService {

    public void viewSummary() {
        System.out.println("Summary (MANAGER/ADMIN)");
    }

    @RoleAllowed({Role.ADMIN}) // stricter for this method
```

```
    public void sensitiveOperation() {
        System.out.println("Sensitive operation executed.");
    }
}

// Authorization gate + invocation helper
class SecurityManager {

    public static Object invokeIfAuthorized(Object target, Method method, Object...
args) throws Throwable {
        // Effective rule: method-level overrides class-level
        RoleAllowed mAnn = method.getAnnotation(RoleAllowed.class);
        RoleAllowed cAnn = target.getClass().getAnnotation(RoleAllowed.class);

        EnumSet<Role> allowed = EnumSet.noneOf(Role.class);
        if (cAnn != null) for (Role r : cAnn.value()) allowed.add(r);
        if (mAnn != null) { allowed.clear(); for (Role r : mAnn.value()) allowed.add(r); }

        Role current = SecurityContext.get();
        if (!allowed.isEmpty() && (current == null || !allowed.contains(current))) {
            throw new IllegalAccessException("Access denied for role " + current + " to
" +
                    target.getClass().getSimpleName() + "#" + method.getName());
        }

        try {
            return method.invoke(target, args);
        } catch (InvocationTargetException ite) {
            throw ite.getTargetException(); // surface real cause
        }
    }
}

// Demo
public class Main {
    public static void main(String[] args) throws Throwable {
        SecureService svc = new SecureService();
        Method summary = SecureService.class.getMethod("viewSummary");
        Method sensitive = SecureService.class.getMethod("sensitiveOperation");

        SecurityContext.set(Role.MANAGER);
        SecurityManager.invokeIfAuthorized(svc, summary);     // OK
        try {
            SecurityManager.invokeIfAuthorized(svc, sensitive); // DENIED
        } catch (IllegalAccessException e) {
            System.out.println(e.getMessage());
        }

        SecurityContext.set(Role.ADMIN);
        SecurityManager.invokeIfAuthorized(svc, sensitive);   // OK
```

```
    }
}
```

Reasoning to interviewer:

- The annotation declares policy right where it matters; the gate enforces it uniformly.
- Enum roles + EnumSet give fast, type-safe checks.
- Method-level override matches how real frameworks (e.g., Spring Security) resolve specificity.
- We fail fast before side effects, and unwrap InvocationTargetException so logs and callers see the true exception.

Future scope:

- Support “anyOf” vs “allOf” semantics, or finer-grained permissions (scopes/claims).
- Plug into a real identity provider (JWT/OIDC) to hydrate SecurityContext.
- Add audit logging on allow/deny and correlation IDs for traceability.
- Cache resolved permissions per method to avoid repeated reflection on hot paths.

Scenario 44: Listing all fields and their annotations in a class

You need an inspection utility that surfaces each field, its type/modifiers, and any annotations attached—handy for documentation, validation, or ORM tooling.

Answer:

I'll use getDeclaredFields() to retrieve every field declared on the class (including private ones). For each field, I'll print its signature and iterate getAnnotations() to list all runtime-retained annotations. If I only care about one annotation, isAnnotationPresent() is faster; otherwise showing all gives a complete picture. For inherited fields, I can walk the superclass chain.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.lang.reflect.Modifier;

// Sample annotations for demo
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD)
@interface Important {}

@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD)
@interface FieldDescription { String info(); String defaultValue() default
"undefined"; }

// Sample class
class Sample {
    @Important
    private String name;

    @FieldDescription(info = "User age", defaultValue = "18")
    int age;

    protected double balance;
}

// Inspector
class FieldInspector {

    public static void inspectClass(Class<?> clazz) {
        System.out.println("Fields in " + clazz.getName() + ":");
        for (Field f : clazz.getDeclaredFields()) {
            String sig = Modifier.toString(f.getModifiers()) + " " +
                    f.getType().getSimpleName() + " " + f.getName();
            System.out.println(" - " + sig);
```

```
            Annotation[] anns = f.getAnnotations(); // all runtime-retained
            if (anns.length == 0) {
                System.out.println("   (no annotations)");
            } else {
                for (Annotation a : anns) {
                    System.out.println("   @" + a.annotationType().getSimpleName() +
details(a));
                }
            }
        }
    }

    private static String details(Annotation a) {
        // Pretty-print common annotations; generic fallback for others
        if (a instanceof FieldDescription fd) {
            return "(info=\"" + fd.info() + "\", defaultValue=\"" + fd.defaultValue() + "\")";
        }
        return "";
    }
}

// Demo
public class FieldInspectorDemo {
    public static void main(String[] args) {
        FieldInspector.inspectClass(Sample.class);
    }
}
```

Reasoning to interviewer:

- getDeclaredFields() captures private/protected/package fields; getFields() would only show public (including inherited).
- Printing annotationType().getSimpleName() plus key attributes makes the output actionable.
- The details() helper shows how you can down-cast to specific annotation types to extract their elements.
- This pattern underpins many tools (doc generators, mappers, validators) that need a full view of a class's shape.

Future scope:

- Walk the superclass hierarchy to include inherited fields.
- Reflect on generic types (e.g., List<String>) via getGenericType() for richer

documentation.

• Cache reflective metadata for performance in long-running apps.

• Emit JSON/Markdown for integration into build pipelines or admin UIs.

Scenario 45: Dynamically invoking private methods for testing

Sometimes you need to poke a private method in a unit test to verify edge behavior without rewriting the production class.

Answer:

I'll locate the method with getDeclaredMethod, flip accessibility with setAccessible(true), and invoke it. I'll unwrap InvocationTargetException to rethrow the real cause so failures are readable. For Java 9+ modules, I'll mention the MethodHandles route as a safer alternative when strong encapsulation blocks setAccessible.

Code:

```
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;

class Hidden {
    private String secretGreet(String name) {
        return "Hi, " + name;
    }
    private static int secretAdd(int a, int b) {
        return a + b;
    }
}

public class PrivateInvoker {
    public static void main(String[] args) throws Exception {
        Hidden obj = new Hidden();

        // 1) Invoke private instance method with argument + return value
        Method greet = Hidden.class.getDeclaredMethod("secretGreet", String.class);
        greet.setAccessible(true);
        try {
            String msg = (String) greet.invoke(obj, "Asha");
            System.out.println(msg);
        } catch (InvocationTargetException ite) {
            throw (ite.getTargetException() instanceof Exception)
                    ? (Exception) ite.getTargetException()
                    : new RuntimeException(ite.getTargetException());
        }

        // 2) Invoke private static method
        Method add = Hidden.class.getDeclaredMethod("secretAdd", int.class,
int.class);
        add.setAccessible(true);
        int sum = (int) add.invoke(null, 4, 5);
```

```
        System.out.println("sum=" + sum);
    }
}

Optional (Java 9+): if modules prevent access, you can use
MethodHandles.privateLookupIn:

// import java.lang.invoke.*;

var lookup = java.lang.invoke.MethodHandles.privateLookupIn(Hidden.class,
java.lang.invoke.MethodHandles.lookup());
var mh = lookup.findVirtual(Hidden.class, "secretGreet",
        java.lang.invoke.MethodType.methodType(String.class, String.class));
String msg = (String) mh.bindTo(new Hidden()).invokeExact("Asha");
```

Reasoning to interviewer:

- getDeclaredMethod lets me target non-public members; setAccessible(true) bypasses visibility just for tests.
- I always unwrap InvocationTargetException so failures show the real exception, not the reflection wrapper.
- For static private methods, pass null as the receiver.
- In modular apps, --add-opens or MethodHandles.privateLookupIn is the correct path when reflective access is denied.

Future scope:

- Prefer testing through public APIs; use private invocation sparingly.
- If private behavior is business-significant, consider extracting it into a package-private collaborator.
- Cache Method/MethodHandle objects for repeated calls to cut reflection overhead.
- Guard tests with assumptions so they don’t fail on stricter module configs in CI.

Scenario 46: Implementing a logger aspect using overridden-method detection

You want to log calls that hit overridden methods (i.e., subclass behavior) to spot polymorphic hotspots.

Answer:

@Override helps at compile time, but it has RetentionPolicy.SOURCE, so you can't read it via reflection at runtime. Instead, I'll detect overridden methods by comparing the target class's implementation with its superclasses: if the method is declared on the subclass and a method with the same signature exists in a superclass, it's an override. For interception without touching the class, I'll use a dynamic proxy around an interface (JDK proxies only proxy interfaces); for concrete classes you'd use CGLIB/ByteBuddy.

Code (interface + proxy + "is overridden" check):

```
import java.lang.reflect.*;
import java.util.Objects;

// 1) Interface so we can use JDK dynamic proxies
interface Displayable {
    void display();
}

// 2) Base class + override
class Parent implements Displayable {
    public void display() { System.out.println("Parent display"); }
}

class Child extends Parent {
    @Override public void display() { System.out.println("Child display"); }
}

// 3) Helper: is this method overridden by target's class relative to some base?
final class Overrides {
    static boolean isOverridden(Class<?> targetClass, Method ifaceMethod,
Class<?> baseClass) {
        try {
            // Implementation method actually executed by target
            Method impl = targetClass.getMethod(ifaceMethod.getName(),
ifaceMethod.getParameterTypes());
            Class<?> decl = impl.getDeclaringClass();

            // Walk up the hierarchy to see if a superclass declares the same signature
            Class<?> cursor = targetClass.getSuperclass();
            while (cursor != null && cursor != Object.class) {
```

```
            try {
                Method superM = cursor.getMethod(ifaceMethod.getName(),
ifaceMethod.getParameterTypes());
                // If the impl is declared on a subclass (decl) and a method exists on a
superclass -> it's an override
                if (!Objects.equals(decl, superM.getDeclaringClass())) {
                    return true;
                }
            } catch (NoSuchMethodException ignore) { /* keep walking */ }
            cursor = cursor.getSuperclass();
        }
        return false;
    } catch (NoSuchMethodException e) {
        return false;
    }
  }
}

// 4) Logging proxy: logs only when call resolves to an overridden implementation
class LoggingProxy implements InvocationHandler {
  private final Object target;
  private final Class<?> baseClass;

  LoggingProxy(Object target, Class<?> baseClass) {
    this.target = target;
    this.baseClass = baseClass;
  }

  @Override
  public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
    boolean overridden = Overrides.isOverridden(target.getClass(), method,
baseClass);
    if (overridden) {
      System.out.println("[LOG] Overridden method called: " +
method.getName() +
                         " on " + target.getClass().getSimpleName());
    }
    try {
      return method.invoke(target, args);
    } catch (InvocationTargetException ite) {
      throw ite.getTargetException();
    }
  }
}

// 5) Demo
public class Main {
  public static void main(String[] args) {
    Displayable child = new Child();
```

```
        Displayable proxy = (Displayable) Proxy.newProxyInstance(
                Displayable.class.getClassLoader(),
                new Class<?>[]{Displayable.class},
                new LoggingProxy(child, Parent.class)
        );

        proxy.display(); // Logs because Child overrides Parent.display()
    }
}
```

Reasoning to interviewer:

- @Override is compile-time assurance; runtime detection relies on the declaring class comparison, not the annotation.
- JDK dynamic proxies require interfaces; for classes, switch to CGLIB/ByteBuddy or Spring AOP.
- Unwrapping InvocationTargetException preserves the original exception for correct error handling.
- The base class parameter makes the intent explicit: “log when behavior differs from this baseline.”

Future scope:

- Extend the matcher to multiple methods/classes, or use annotations (@LogExecution) to select which ones to watch.
- Add argument/result logging and timing around overridden calls for performance profiling.
- Cache override checks per (class, method) to avoid repeated reflection on hot paths.
- For broad coverage in a codebase, weave the aspect at build-time or agent-time with ByteBuddy.

Scenario 47: Custom annotation with multiple parameters and defaults

You want one annotation that carries a few knobs (like name, version, enabled), with sane defaults, and the ability to read those values at runtime.

Answer:

I'll declare an annotation @Config with three elements: a required name(), plus version() and enabled() with defaults. I'll keep retention at RUNTIME so reflection can read it, and I'll add @Inherited so subclasses pick up a class-level configuration by default. At runtime, I'll access the annotation via getAnnotation, handle the "annotation missing" case cleanly, and demonstrate how defaults flow when elements are not provided.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.AnnotatedElement;

// 1) Define the annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@Inherited // subclasses inherit the class-level annotation
@interface Config {
    String name();              // required
    String version() default "1.0";
    boolean enabled() default true;
}

// 2) Apply it
@Config(name = "MyModule", version = "2.1") // enabled defaults to true
class Module {}

class ChildModule extends Module {} // inherits @Config because of @Inherited

// 3) Reader utilities
class ConfigUtil {
    static void printConfig(AnnotatedElement element) {
        Config cfg = element.getAnnotation(Config.class);
        if (cfg == null) {
            System.out.println(element + " has no @Config");
            return;
        }
        System.out.println("name=" + cfg.name() +
                           ", version=" + cfg.version() +
                           ", enabled=" + cfg.enabled());
    }
```

```
}

// 4) Demo
public class AnnotationReader {
    public static void main(String[] args) {
        ConfigUtil.printConfig(Module.class);       // name=MyModule, version=2.1,
enabled=true
        ConfigUtil.printConfig(ChildModule.class);  // inherits same values
        ConfigUtil.printConfig(String.class);       // no @Config
    }
}
```

Reasoning to interviewer:

- Defaults live in the annotation type, so callers only set non-default values; everything else is consistent and centralized.
- @Inherited lets class-level metadata flow down to subclasses, which is handy for broad module settings.
- Always guard against a missing annotation (null from getAnnotation) to avoid NPEs.
- Keeping retention at RUNTIME is essential—otherwise reflection won't see the annotation.

Future scope:

- Add an enum for stage (DEV/QA/PROD) to avoid stringly-typed mistakes.
- Make the annotation repeatable if you need multiple named configs: @Repeatable(Configs.class).
- Provide a small loader that merges class-level @Config with method-level overrides (if you expand the target later).
- Cache annotation lookups per class to avoid repeated reflection on hot paths.

Scenario 48: Suppressing specific warnings in a legacy codebase

You want to quiet only the noise you've audited (raw types, unchecked casts, deprecation), without muting valuable warnings elsewhere.

Answer:

I'll use @SuppressWarnings with precise keys like "unchecked", "rawtypes", "deprecation", or "removal" (for JDK APIs marked for removal). I'll apply it at the narrowest scope possible—preferably the method or even a local variable—so the rest of the class still emits useful diagnostics. For varargs with generics, I'll prefer @SafeVarargs on final/static/private methods to silence heap-pollution warnings correctly.

Code:

```
import java.util.ArrayList;
import java.util.List;

class LegacyBits {

    // 1) Narrow suppression at method level (preferred over class-level blanket)
    @SuppressWarnings({"unchecked", "rawtypes"})
    void useRawList() {
        List raw = new ArrayList();  // raw type
        raw.add("Test");
        // An explicit, audited cast:
        List<String> names = (List<String>) raw; // unchecked cast
        System.out.println(names.get(0));
    }

    // 2) Local suppression via variable (very narrow scope)
    void localSuppression() {
        @SuppressWarnings("rawtypes")
        List temp = new ArrayList(); // raw only for this variable
        temp.add(123);
        System.out.println(temp);
    }

    // 3) Deprecation / removal (JDK 9+ can emit “removal” warnings)
    @SuppressWarnings({"deprecation", "removal"})
    void useDeprecatedApi() {
        DeprecatedApi.old(); // suppose annotated with @Deprecated(forRemoval =
true)
    }

    // 4) Generic varargs: prefer @SafeVarargs where allowed
    @SafeVarargs
```

```
    private final <T> List<T> concat(List<T>... lists) {
        List<T> out = new ArrayList<>();
        for (List<T> l : lists) out.addAll(l);
        return out;
    }
}

// Simulated deprecated API
class DeprecatedApi {
    @Deprecated(forRemoval = true, since = "2.0")
    static void old() { System.out.println("old API"); }
}
```

Reasoning to interviewer:

- Use the smallest applicable scope to minimize the “mute” footprint. Method-level beats class-level.
- Choose specific keys; avoid broad or incorrect ones. Common keys: "unchecked", "rawtypes", "deprecation", "removal", "serial", "fallthrough", "cast".
- Prefer @SafeVarargs (when eligible) over blanket suppression for generic varargs—this documents intent and is safer.
- Keep suppression close to the code that needs it so future refactors don’t accidentally hide new issues.

Future scope:

- Pair suppressions with comments linking to a tracking ticket to revisit and remove them.
- Add build-time checks (Error Prone/Checkstyle) to forbid class-level suppressions except in whitelisted packages.
- Gradually replace raw types with generics and deprecated calls with supported alternatives, shrinking the number of suppressions over time.

Scenario 49: Creating an annotation processor to generate documentation

You want compile-time tooling that walks annotated code and emits human-readable docs (e.g., Markdown) without any runtime cost.

Answer:

I'll define a source-retention annotation (e.g., @DocField) and an annotation processor that extends AbstractProcessor. In process, I'll collect all fields tagged with @DocField, group them by the enclosing class, and write a docs/<Class>.md file via the Filer. I'll also use Messager for compile-time feedback and register the processor under META-INF/services/javax.annotation.processing.Processor.

Code:

```
// --- api/src/main/java/com/example/doc/DocField.java
package com.example.doc;

import java.lang.annotation.*;

@Retention(RetentionPolicy.SOURCE)          // compile-time only; no runtime
overhead
@Target(ElementType.FIELD)
public @interface DocField {
    String description();
    String example() default "";
    String since() default "1.0";
}
// --- app/src/main/java/com/example/model/Product.java
package com.example.model;

import com.example.doc.DocField;

public class Product {
    @DocField(description = "Human-friendly product name", example = "Roadster
X")
    private String name;

    @DocField(description = "Unit price in USD", example = "999.99", since = "1.2")
    private double price;

    private String internalCode; // not documented
}
// --- processor/src/main/java/com/example/processor/DocProcessor.java
package com.example.processor;

import com.example.doc.DocField;
import javax.annotation.processing.*;
```

```
import javax.lang.model.SourceVersion;
import javax.lang.model.element.*;
import javax.lang.model.util.ElementFilter;
import javax.tools.Diagnostic;
import javax.tools.FileObject;
import javax.tools.StandardLocation;
import java.io.IOException;
import java.io.Writer;
import java.util.*;

@SupportedAnnotationTypes("com.example.doc.DocField")
@SupportedSourceVersion(SourceVersion.RELEASE_17)
public class DocProcessor extends AbstractProcessor {

    @Override
    public boolean process(Set<? extends TypeElement> annotations,
RoundEnvironment roundEnv) {
        if (annotations.isEmpty()) return false;

        // Group fields by enclosing class
        Map<TypeElement, List<VariableElement>> byClass = new HashMap<>();
        for (Element e : roundEnv.getElementsAnnotatedWith(DocField.class)) {
            if (e.getKind() != ElementKind.FIELD) continue;
            VariableElement field = (VariableElement) e;
            TypeElement owner = (TypeElement) field.getEnclosingElement();
            byClass.computeIfAbsent(owner, k -> new ArrayList<>()).add(field);
        }

        for (Map.Entry<TypeElement, List<VariableElement>> entry :
byClass.entrySet()) {
            TypeElement owner = entry.getKey();
            List<VariableElement> fields = entry.getValue();
            try {
                writeMarkdown(owner, fields);
                processingEnv.getMessager().printMessage(
                        Diagnostic.Kind.NOTE,
                        "Generated docs for " + owner.getQualifiedName()
                );
            } catch (IOException ex) {
                processingEnv.getMessager().printMessage(
                        Diagnostic.Kind.ERROR,
                        "Failed to write docs: " + ex.getMessage(), owner
                );
            }
        }
        // Claiming we processed DocField
        return true;
    }
```

```
    private void writeMarkdown(TypeElement owner, List<VariableElement> fields)
throws IOException {
        String pkg =
processingEnv.getElementUtils().getPackageOf(owner).getQualifiedName().toStrin
g();
        String simpleName = owner.getSimpleName().toString();
        String fileName = "docs/" + pkg + "." + simpleName + ".md";

        Filer filer = processingEnv.getFiler();
        FileObject file = filer.createResource(StandardLocation.CLASS_OUTPUT, "",
fileName, owner);

        try (Writer w = file.openWriter()) {
            w.write("# " + simpleName + " fields\n\n");
            w.write("| Field | Type | Description | Example | Since |\n");
            w.write("|------:|:-----|:-----------|:--------|:------|\n");
            // Only include fields actually annotated
            for (VariableElement f : fields) {
                DocField df = f.getAnnotation(DocField.class); // ok for SOURCE
retention in processors
                String type = f.asType().toString();
                w.write("| " + f.getSimpleName() + " | " + type + " | "
                        + escape(df.description()) + " | "
                        + (df.example().isEmpty() ? "—" : code(df.example())) + " | "
                        + df.since() + " |\n");
            }
        }
    }

    private String escape(String s) { return s.replace("|", "\\|"); }
    private String code(String s) { return "`" + s.replace("`", "\\`") + "`"; }
}
# --- processor/src/main/resources/META-
INF/services/javax.annotation.processing.Processor
com.example.processor.DocProcessor
```

Reasoning to interviewer:

- SOURCE retention keeps annotations out of bytecode and avoids runtime cost—perfect for doc generation.
- Grouping by enclosing type gives one page per class, which mirrors how devs think about models.
- Writing to CLASS_OUTPUT ensures generated docs land alongside compiled artifacts and can be captured by the build.
- I rely on Messager for actionable compiler feedback (errors/warnings/notes).

Future scope:

- Support class-level docs and method parameters (@DocParam).
- Emit multiple formats (Markdown/JSON/OpenAPI snippets).
- Add @Repeatable for multi-notes, and support field deprecation with migration hints.
- Make it incremental and deterministic (stable ordering) to play well with Gradle caching.

Scenario 50: Dynamic proxy for logging method calls of an interface implementation

You want zero-touch logging around interface methods: who was called, with which args, how long it took, and what happened.

Answer:

I'll wrap the real implementation with a JDK dynamic proxy and an InvocationHandler that logs the method name/args, times the call, unwraps and rethrows exceptions, and returns the result. I'll also handle equals/hashCode/toString explicitly so the proxy behaves like the original object.

Code:

```
import java.lang.reflect.*;
import java.util.Arrays;

// --- Target API
interface Service {
    void doWork(String jobId);
    int compute(int x, int y);
}

class ServiceImpl implements Service {
    public void doWork(String jobId) {
        System.out.println("Work done for " + jobId + ".");
    }
    public int compute(int x, int y) { return x + y; }
}

// --- Logging handler
class LoggingHandler implements InvocationHandler {
    private final Object target;

    LoggingHandler(Object target) { this.target = target; }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
        // Handle Object methods explicitly
        if (method.getDeclaringClass() == Object.class) {
            return switch (method.getName()) {
                case "toString" -> "LoggingProxy(" + target + ")";
                case "hashCode" -> System.identityHashCode(proxy);
                case "equals"   -> proxy == args[0];
                default -> method.invoke(target, args);
            };
```

```
        }

        String argStr = (args == null) ? "[]" : Arrays.toString(args);
        long startNs = System.nanoTime();
        System.out.printf("[ENTER] %s %s%n", method.getName(), argStr);
        try {
            Object result = method.invoke(target, args);
            long tookMs = (System.nanoTime() - startNs) / 1_000_000;
            System.out.printf("[EXIT ] %s took=%dms result=%s%n",
method.getName(), tookMs, String.valueOf(result));
            return result;
        } catch (InvocationTargetException ite) {
            long tookMs = (System.nanoTime() - startNs) / 1_000_000;
            Throwable cause = ite.getTargetException();
            System.out.printf("[ERROR] %s took=%dms ex=%s%n",
method.getName(), tookMs, cause);
            throw cause; // preserve original semantics
        }
    }
}

// --- Factory
class Proxies {
    @SuppressWarnings("unchecked")
    static <T> T logCalls(T target, Class<T> iface) {
        return (T) Proxy.newProxyInstance(
                iface.getClassLoader(),
                new Class<?>[]{ iface },
                new LoggingHandler(target)
        );
    }
}

// --- Demo
public class Main {
    public static void main(String[] args) {
        Service svc = new ServiceImpl();
        Service logged = Proxies.logCalls(svc, Service.class);

        logged.doWork("JOB-7");
        int sum = logged.compute(5, 6);
        System.out.println("sum=" + sum);
    }
}
```

Reasoning to interviewer:

• JDK proxies work for interfaces; the handler centralizes cross-cutting logging without touching business code.

• I unwrap InvocationTargetException so callers see the real exception type and stack.

• Guarding equals/hashCode/toString prevents odd behavior in maps/logging.

• The factory is generic and easy to reuse across interfaces.

Future scope:

• For concrete classes, switch to ByteBuddy/CGLIB or use Spring AOP.

• Add argument redaction for sensitive data and structured logging (JSON) with correlation IDs.

• Enrich with metrics (timers/counters) and tracing headers for distributed systems.

• Cache Method metadata and allow opt-in filters (e.g., log only methods annotated with @LogExecution).

Scenario 51: Modifying field values dynamically for configuration

Sometimes you need to flip a private switch—API keys, feature flags—without adding setters or changing the class's public API.

Answer:

I'll locate the field by name (walking the class hierarchy to handle inherited fields), call setAccessible(true) to bypass visibility, and set the value. I'll add basic type checks so we don't stuff a String into an int. For static fields, I'll pass null as the instance. Note that changing final fields isn't reliable post–Java 9; I'll avoid that. If you're in a modular JDK, you may need --add-opens your.module/package=ALL-UNNAMED.

Code:

```
import java.lang.reflect.Field;

class Config {
    private String apiKey = "default";
    private int timeoutMs = 1000;

    @Override public String toString() {
        return "Config{apiKey='" + apiKey + "', timeoutMs=" + timeoutMs + "}";
    }
}

public class FieldModifier {

    public static void setField(Object target, String fieldName, Object newValue) {
        if (target == null) throw new IllegalArgumentException("target is null");
        Class<?> cls = target.getClass();
        Field field = findField(cls, fieldName);
        if (field == null) throw new IllegalArgumentException("No field '" + fieldName +
"' in " + cls);

        if (java.lang.reflect.Modifier.isFinal(field.getModifiers())) {
            throw new UnsupportedOperationException("Refusing to set final field: " +
fieldName);
        }

        field.setAccessible(true);
        Class<?> ft = field.getType();

        try {
            // Basic type compatibility guard (auto-unboxing works if wrappers match
primitives)
            if (newValue != null && !wrap(ft).isAssignableFrom(newValue.getClass())) {
```

```
                throw new IllegalArgumentException("Incompatible value type: " +
newValue.getClass() + " for " + ft);
            }
            field.set(target, newValue);
        } catch (IllegalAccessException e) {
            throw new RuntimeException("Failed to set field: " + fieldName, e);
        }
    }

    private static Field findField(Class<?> type, String name) {
        Class<?> c = type;
        while (c != null && c != Object.class) {
            try { return c.getDeclaredField(name); }
            catch (NoSuchFieldException ignored) { c = c.getSuperclass(); }
        }
        return null;
    }

    private static Class<?> wrap(Class<?> t) {
        if (!t.isPrimitive()) return t;
        if (t == int.class) return Integer.class;
        if (t == long.class) return Long.class;
        if (t == boolean.class) return Boolean.class;
        if (t == double.class) return Double.class;
        if (t == float.class) return Float.class;
        if (t == char.class) return Character.class;
        if (t == short.class) return Short.class;
        if (t == byte.class) return Byte.class;
        return t;
    }

    public static void main(String[] args) {
        Config cfg = new Config();
        System.out.println(cfg);

        setField(cfg, "apiKey", "newApiKey123");
        setField(cfg, "timeoutMs", 2500);

        System.out.println(cfg); // Config{apiKey='newApiKey123', timeoutMs=2500}
    }
}
```

Reasoning to interviewer:

- Walking the hierarchy makes it work even when fields live on a superclass.
- Guarding types prevents subtle ClassCast issues at runtime.
- I avoid final updates—they're JVM- and version-sensitive.

• If module boundaries block setAccessible, use --add-opens in tests or switch to MethodHandles with privateLookupIn.

Future scope:

• Add getters/setters via MethodHandles for faster repeated access.
• Support batch updates from a Map<String,Object> or config file.
• Add thread-safety if you mutate shared config live (e.g., volatile fields or atomic references).
• Validate values against constraints (e.g., @MaxLen, @NonNull) before writing.

Scenario 52: Detecting and invoking methods annotated with a custom @Init

Think of this like a lightweight lifecycle hook—methods you want run automatically before use.

Answer:

I'll define @Init (optionally with an order to control sequence), scan declared methods (and superclasses), filter to zero-arg methods marked with @Init, sort by order, setAccessible(true) for non-public hooks, and invoke them. Any exception should fail fast so you don't run with a half-initialized object.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Init {
    int order() default 0; // lower runs first
}

class BaseService {
    @Init(order = -1)
    protected void baseWarmup() {
        System.out.println("Base warmup");
    }
}

class Service extends BaseService {
    @Init
    public void initialize() {
        System.out.println("Initialization logic");
    }

    public void run() {
        System.out.println("Running");
    }

    @Init(order = 5)
    private void lateStep() {
        System.out.println("Late step");
    }
```

```
}

class InitInvoker {

    public static void runInits(Object target) {
        if (target == null) return;
        List<Method> initMethods = findInitMethods(target.getClass());

        // Sort by order, stable for deterministic runs
        initMethods.sort(Comparator.comparingInt(m ->
m.getAnnotation(Init.class).order()));

        for (Method m : initMethods) {
            if (m.getParameterCount() != 0) {
                throw new IllegalStateException("@Init methods must be no-arg: " + m);
            }
            m.setAccessible(true);
            try {
                m.invoke(target);
            } catch (ReflectiveOperationException e) {
                // unwrap InvocationTargetException if needed
                Throwable cause = (e.getCause() != null) ? e.getCause() : e;
                throw new RuntimeException("Init failed in " + m.getName() + ": " +
cause, cause);
            }
        }
    }

    private static List<Method> findInitMethods(Class<?> type) {
        List<Method> out = new ArrayList<>();
        Class<?> c = type;
        while (c != null && c != Object.class) {
            for (Method m : c.getDeclaredMethods()) {
                if (m.isAnnotationPresent(Init.class)) {
                    out.add(m);
                }
            }
            c = c.getSuperclass();
        }
        return out;
    }
}

public class InitDemo {
    public static void main(String[] args) {
        Service service = new Service();
        InitInvoker.runInits(service);
        service.run();
    }
}
```

Reasoning to interviewer:

- Ordering avoids “who runs first?” uncertainty, which matters when one init depends on another.
- Scanning superclasses lets base classes define shared lifecycle hooks.
- Enforcing zero-arg methods keeps the contract simple and invocation safe.
- Deterministic sort + fail-fast on exceptions gives predictable startup behavior.

Future scope:

- Add @PostInit and @PreDestroy hooks to mirror container lifecycles.
- Allow async init steps and collect futures before moving to run().
- Cache method lookups per class to reduce reflection overhead if you construct many instances.
- Integrate with dependency injection so @Inject runs before @Init.

Scenario 53: Runtime analysis of class method signatures and annotations

You want a small introspection utility that prints each method's signature (modifiers, return type, parameters) and the annotations on the method and its parameters.

Answer:

I'll iterate getDeclaredMethods() to include non-public members, filter out synthetic/bridge methods to avoid noise from generics, and print details: modifiers, return type, name, parameters (with types and parameter-level annotations), declared exceptions, and any method annotations. If you compile with -parameters, I can also show real parameter names.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.Arrays;

@Retention(RetentionPolicy.RUNTIME)
@interface Info { String description(); }

class Sample {
    @Info(description = "Adds two numbers")
    public int compute(int x, int y) throws IllegalArgumentException {
        return x + y;
    }

    @Deprecated
    protected <T extends Number> T echo(T value) { return value; }

    private void hidden() {}
}

public class MethodInspector {
    public static void main(String[] args) {
        inspectClass(Sample.class);
    }

    static void inspectClass(Class<?> clazz) {
        System.out.println("Inspecting: " + clazz.getName());
        for (Method m : clazz.getDeclaredMethods()) {
            if (m.isSynthetic() || m.isBridge()) continue; // skip compiler artifacts

            // Signature header
            String mods = Modifier.toString(m.getModifiers());
            String ret  = m.getGenericReturnType().getTypeName();
```

```
            System.out.println("\n" + mods + " " + ret + " " + m.getName() +
signature(m));

            // Method-level annotations
            for (Annotation ann : m.getDeclaredAnnotations()) {
                System.out.println("  @" + ann.annotationType().getSimpleName() +
details(ann));
            }

            // Parameter annotations
            Parameter[] params = m.getParameters();
            for (int i = 0; i < params.length; i++) {
                Parameter p = params[i];
                if (p.getAnnotations().length == 0) continue;
                System.out.print("  param[" + i + "] " +
                        (p.isNamePresent() ? p.getName() : ("arg" + i)) + " anns: ");
                System.out.println(Arrays.toString(
                        Arrays.stream(p.getAnnotations())
                              .map(a -> "@" + a.annotationType().getSimpleName())
                              .toArray()
                ));
            }

            // Declared exceptions
            if (m.getExceptionTypes().length > 0) {
                System.out.println("  throws: " +
Arrays.toString(m.getExceptionTypes()));
            }
        }
    }

    // Build a readable parameter list with generics
    private static String signature(Method m) {
        String params = Arrays.stream(m.getGenericParameterTypes())
                              .map(Type::getTypeName)
                              .reduce((a,b) -> a + ", " + b).orElse("");
        return "(" + params + ")";
    }

    // Pretty-print annotation details for known types
    private static String details(Annotation a) {
        if (a instanceof Info info) return "(description=\"" + info.description() + "\")";
        if (a instanceof Deprecated) return "(since=legacy)";
        return "";
    }
}
```

Reasoning to interviewer:

• getDeclaredMethods() sees private methods; I skip synthetic/bridge to avoid compiler-generated noise.
• Using getGenericReturnType()/getGenericParameterTypes() preserves type arguments for richer output.
• Parameter names require the -parameters compiler flag; otherwise I fall back to arg0, arg1.
• Printing parameter-level annotations highlights validation/security metadata used by frameworks.

Future scope:

• Walk the superclass/interface hierarchy to include inherited methods.
• Export the info as JSON/Markdown for documentation pipelines.
• Add filters (e.g., only methods annotated with @Info).
• Cache reflective metadata for performance in large codebases.

Scenario 54: Using reflection to handle multiple constructors and choose one at runtime

You need to construct an object when argument types are known only at runtime, and you might have multiple constructors to choose from.

Answer:

I'll search the constructors, find the best match for the runtime arguments (considering primitive/wrapper compatibility and varargs), make it accessible if needed, and instantiate. I'll unwrap InvocationTargetException so callers see the real cause. If you know the exact signature, getConstructor(...) is fastest; otherwise a "best fit" resolver is safer.

Code:

```
import java.lang.reflect.Constructor;
import java.lang.reflect.InvocationTargetException;
import java.util.Arrays;

class Person {
    private final String name;
    private final int age;

    public Person() { this("Unknown", 0); }
    public Person(String name) { this(name, 0); }
    public Person(String name, int age) { this.name = name; this.age = age; }

    @Override public String toString() { return "Person{name='" + name + "', age="
+ age + "}"; }
}

public class ConstructorInvoker {

    public static void main(String[] args) {
        Person p1 = create(Person.class, new Object[]{});               // default
        Person p2 = create(Person.class, new Object[]{"Alice"});        // (String)
        Person p3 = create(Person.class, new Object[]{"Bob", 30});      // (String,int)
        System.out.println(p1);
        System.out.println(p2);
        System.out.println(p3);
    }

    // Best-effort constructor selection
    public static <T> T create(Class<T> type, Object[] args) {
        try {
            if (args == null) args = new Object[0];
            Constructor<?>[] ctors = type.getDeclaredConstructors();
```

```
            Constructor<?> best = null;
            int bestScore = Integer.MIN_VALUE;

            for (Constructor<?> c : ctors) {
                Class<?>[] params = c.getParameterTypes();
                if (!isArityCompatible(params, args)) continue;

                int score = matchScore(params, args);
                if (score > bestScore) { bestScore = score; best = c; }
            }
            if (best == null) {
                throw new NoSuchMethodException("No compatible constructor for " +
type.getName() +
                        " with args " + Arrays.toString(Arrays.stream(args).map(a ->
a==null? "null" : a.getClass().getName()).toArray()));
            }

            if (!best.canAccess(null)) best.setAccessible(true);
            @SuppressWarnings("unchecked")
            T instance = (T) best.newInstance(args);
            return instance;

        } catch (InvocationTargetException ite) {
            Throwable cause = ite.getTargetException();
            throw new RuntimeException("Constructor threw: " + cause, cause);
        } catch (ReflectiveOperationException e) {
            throw new RuntimeException(e);
        }
    }

    private static boolean isArityCompatible(Class<?>[] params, Object[] args) {
        if (params.length != args.length) return false;
        return true;
    }

    // Higher score = better match (exact > wrapper/primitive > upcast > null
assignable)
    private static int matchScore(Class<?>[] params, Object[] args) {
        int score = 0;
        for (int i = 0; i < params.length; i++) {
            Class<?> p = params[i];
            Object a = args[i];
            if (a == null) {
                if (p.isPrimitive()) return Integer.MIN_VALUE; // cannot assign null to
primitive
                score += 1; // nullable reference
                continue;
            }
            Class<?> ac = a.getClass();
            if (p.isPrimitive()) {
```

```
                if (wrap(p).equals(ac)) score += 3;          // exact via boxing
                else score += isWideningBoxed(p, ac) ? 2 : -1000;
            } else {
                if (p.equals(ac)) score += 4;               // exact
                else if (p.isAssignableFrom(ac)) score += 2; // upcast
                else score -= 1000;                         // incompatible
            }
        }
        return score;
    }

    private static boolean isWideningBoxed(Class<?> primitiveParam, Class<?>
argClass) {
        // Simple example: allow Integer -> long parameter, etc.
        if (primitiveParam == long.class && argClass == Integer.class) return true;
        if (primitiveParam == double.class && (argClass == Float.class || argClass ==
Long.class || argClass == Integer.class)) return true;
        if (primitiveParam == float.class && (argClass == Integer.class || argClass ==
Long.class)) return true;
        return false;
    }

    private static Class<?> wrap(Class<?> p) {
        if (!p.isPrimitive()) return p;
        if (p == int.class) return Integer.class;
        if (p == long.class) return Long.class;
        if (p == boolean.class) return Boolean.class;
        if (p == double.class) return Double.class;
        if (p == float.class) return Float.class;
        if (p == char.class) return Character.class;
        if (p == short.class) return Short.class;
        if (p == byte.class) return Byte.class;
        return p;
    }
}
```

Reasoning to interviewer:

- If you know the signature, getConstructor(String.class, int.class) is direct and fastest.
- For dynamic inputs, a scoring matcher avoids brittle instanceof chains and handles boxing/widening.
- setAccessible(true) covers non-public constructors in tests; in modular apps, you may need --add-opens.
- Unwrapping InvocationTargetException preserves the original failure type/stack for debugging.

Future scope:

- Support varargs constructors by detecting isVarArgs() and packing trailing args.
- Prefer Constructor::newInstance or MethodHandles.Lookup for faster repeated construction.
- Add parameter name/value mapping to construct from a config map.
- Cache the chosen constructor per (class, arg signature) to reduce reflection overhead.

Scenario 55: Creating a custom annotation to mark deprecated fields

You want a declarative way to flag fields as deprecated (with guidance) and surface warnings whenever code touches them via reflection-driven access.

Answer:

I'll define @DeprecatedField with optional metadata (since, replacement, and severity). A small reflection helper will centralize all reflective field reads/writes. Before accessing a field, it checks for @DeprecatedField and logs a warning (or throws) with a clear migration hint. This keeps deprecation policy in one place and avoids silent usage.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.Field;

// 1) Annotation: add useful metadata beyond the built-in @Deprecated
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface DeprecatedField {
    String since() default "";
    String replaceWith() default "";
    boolean error() default false; // escalate to exception if true
}

// 2) Example domain
class Config {
    @DeprecatedField(since = "2.3", replaceWith = "newSetting", error = false)
    public String oldSetting = "old";

    public String newSetting = "new";

    private int timeoutMs = 500; // show private field access too
}

// 3) Centralized reflective accessor with deprecation checks
class FieldAccess {

    public static Object get(Object target, String fieldName) {
        Field f = findField(target.getClass(), fieldName);
        checkDeprecated(target.getClass(), f, "read");
        f.setAccessible(true);
        try {
            return f.get(target);
        } catch (IllegalAccessException e) {
```

```
            throw new RuntimeException("Cannot read field: " + f.getName(), e);
        }
    }

    public static void set(Object target, String fieldName, Object value) {
        Field f = findField(target.getClass(), fieldName);
        checkDeprecated(target.getClass(), f, "write");
        f.setAccessible(true);
        try {
            // simple compatibility guard
            if (value != null && !wrap(f.getType()).isAssignableFrom(value.getClass()))
{
                throw new IllegalArgumentException("Incompatible value type: " +
                        value.getClass().getName() + " for field " + f.getName() +
                        " of type " + f.getType().getName());
            }
            f.set(target, value);
        } catch (IllegalAccessException e) {
            throw new RuntimeException("Cannot write field: " + f.getName(), e);
        }
    }

    private static void checkDeprecated(Class<?> owner, Field f, String action) {
        DeprecatedField df = f.getAnnotation(DeprecatedField.class);
        if (df == null) return;

        String msg = String.format(
                "Warning: %s deprecated field %s.%s (since=%s, replaceWith=%s)",
                action, owner.getSimpleName(), f.getName(),
                empty(df.since(), "n/a"),
                empty(df.replaceWith(), "n/a")
        );

        if (df.error()) {
            throw new IllegalStateException(msg);
        } else {
            System.out.println(msg);
        }
    }

    private static String empty(String s, String fallback) { return (s == null ||
s.isBlank()) ? fallback : s; }

    private static Field findField(Class<?> type, String name) {
        Class<?> c = type;
        while (c != null && c != Object.class) {
            try { return c.getDeclaredField(name); }
            catch (NoSuchFieldException ignored) { c = c.getSuperclass(); }
        }
```

```
        throw new IllegalArgumentException("No field '" + name + "' on " +
type.getName());
    }

    private static Class<?> wrap(Class<?> t) {
        if (!t.isPrimitive()) return t;
        if (t == int.class) return Integer.class;
        if (t == long.class) return Long.class;
        if (t == boolean.class) return Boolean.class;
        if (t == double.class) return Double.class;
        if (t == float.class) return Float.class;
        if (t == char.class) return Character.class;
        if (t == short.class) return Short.class;
        if (t == byte.class) return Byte.class;
        return t;
    }
}

// 4) Demo
public class FieldUsageDetector {
    public static void main(String[] args) {
        Config config = new Config();

        // Reading deprecated public field
        Object oldVal = FieldAccess.get(config, "oldSetting"); // warns
        System.out.println("oldSetting = " + oldVal);

        // Writing deprecated public field
        FieldAccess.set(config, "oldSetting", "legacy"); // warns

        // Reading non-deprecated public field
        System.out.println("newSetting = " + FieldAccess.get(config, "newSetting"));

        // Accessing private field still runs through the same guard
        System.out.println("timeoutMs = " + FieldAccess.get(config, "timeoutMs"));
        FieldAccess.set(config, "timeoutMs", 750);
        System.out.println("timeoutMs = " + FieldAccess.get(config, "timeoutMs"));
    }
}
```

Reasoning to interviewer:

- The annotation adds migration context (since, replaceWith) that teams actually need during cleanup.
- Centralizing reflective access guarantees consistent warnings across reads and writes (and across private/public fields).
- I walk the class hierarchy, so inherited fields are covered too.

• The error flag escalates usage from a warning to an exception when you’re ready to harden the policy.

Future scope:

• Also tag fields with built-in @Deprecated so IDEs and compilers nudge developers even outside reflection.
• Add a compile-time annotation processor to fail builds if deprecated fields are referenced directly in source.
• Emit structured logs (JSON) with call-site info, or integrate with a metrics counter to track remaining usages.
• If you need to catch non-reflective accesses at runtime, use bytecode instrumentation (ByteBuddy/Java agent) to weave guards.